
MoE Router distribution exploration

Date: January 20, 2026

Andres Nowak¹

1. Introduction

Recent work in zero-computation experts; (Team et al., 2025), (Zeng et al., 2024), (Jin et al., 2024) and my experiments, and work in very sparse MoEs trinity large (Singh et al., 2026) (where they have a very sparse model with top-k of 4, so only around 4% of the parameters are used per token), motivated an investigation into whether MoEs already learn a sparse representation, meaning that many tokens effectively utilize fewer than K experts.¹

When training MoE's, if we aren't using only Auxiliary-free balancing loss, we modify the router scores based on a uniform distribution that we want to force across experts in a Micro-Batch during pre-training 1. So this loss changes how the score of the router will behave, but if in a micro-batch at inference we only give one batch topic, then we can get scores that aren't uniform and what if maybe are we always forcing a top-k distribution when the model for many tokens maybe wants to use less.

$$\begin{aligned}\mathcal{L}_{\text{aux_bal_loss}} &= \frac{N}{k(T_{\text{seq}} \cdot B)^2} \sum_{i=1}^N (P_i f_i) \\ f_i &= \sum_{x \in (T_{\text{seq}} \cdot B)} \text{routing_map}(x, i), \quad f_i \in \mathbb{N}_0^N \\ P_i &= \sum_{x \in (T_{\text{seq}} \cdot B)} \text{probs}(x, i), \quad P_i \in \mathbb{R}^N\end{aligned}$$

We therefore examine the impact of deactivating experts within the top- k set based on a simple score threshold p , measuring the resulting model performance to quantify potential redundancy in expert selection.

2. Router exploration

The router distributions we will visualize come from the probability distribution from the experts when they are inside the top- k selection 2 for a token. Here we want to visualize the value score that is being used in the weighted sum of the experts, to visualize, **at least based on the router**

side, how much weight are we giving an expert for a token.

$$\begin{aligned}g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases} \\ s &= \sigma(W_g x)\end{aligned}$$

where W_g is the routers weight, σ is an activation function like sigmoid or softmax, and $s_{i,t}$ means if the expert i is chosen for token t .

2.1. Dataset-Specific Input Processing

We visualize MMLU, MMLU-Pro, Hellaswag, ARC and Winogrande dataset they will be used to evaluate the models and test the theory of what happens if we deactivate some experts in the top-k based on score threshold.

What we do for each dataset:

- **MMLU and MMLU-Pro:** We only pass the question as input to the model, and that is what we visualize here with the router distributions (the same input that `lm_eval` (Gao et al., 2024) will pass).
- **Hellaswag:** We only pass the context (`ctx_a + ctx_b`), and those are the router distributions we visualize (the same input that `lm_eval` (Gao et al., 2024) will pass).
- **AllenAI ARC:** We only pass the question as input, and those are the router distributions we visualize. (In `lm_eval` (Gao et al., 2024) they pass `'Question: {{question}}\nAnswer: '`). We visualize both challenge and easy splits together.
- **Winogrande:** We take everything before `'_'` and then add the option (we create one input for each option). This is the same as in `lm_eval` (Gao et al., 2024), and while it may not have been strictly necessary to add the option (just the text before `'_'` would suffice), we do this to show the router distributions that should be occurring during evaluations.
- **All datasets:** We do not visualize the router scores for padding tokens.

¹Code `github` url: <https://github.com/andresnowak/moe-router-exploration>

2.2. Purpose of Visualizations

We perform these visualizations to understand what we are doing when deactivating all experts in the top-k that have a score below a threshold. With this, we can see we are affecting a lot of experts, understand what thresholds are important, what layers are affecting the most, and also observe what score values the router decides to assign to experts in the top-k.

2.3. Model Selection

Note: All models we are testing are base models except for GPT-OSS (which does not have a base model).

3. Visualizations

3.1. Deepseek-MoE

First Deepseek MoE has a Top-k = 6, $N = 64$ experts, 2 shared experts, 1 dense layer and 28 layers in total (<https://huggingface.co/deepseek-ai/deepseek-moe-16b-base/blob/main/config.json>).

For DeepSeek-MoE (Dai et al., 2024), the MoE architecture and balancing method that was used is:

$$h'_t = x_t + \sum_{i=1}^{N_r} \text{FFN}_i(x_t) + \sum_{i=1}^{N_s} \text{FFN}_i(x_t)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leq j \leq N_r\}, K), \\ 0, & \text{otherwise,} \end{cases}$$

$$s_{i,t} = \text{Softmax}_i(W_i x_t^T)$$

where x_t is the input, W_i^g is the vector for expert _{i} from the routers weight W_g , N_r is the number of routable experts, N_s is the number of shared experts (experts that are always used for every token). The method a softmax over the logits and then we do a top-k without a renormalization to get the K chosen experts for token t , then we do a weighted sum with the results of each FFN_i expert and then the new hidden state for token x_t with the residual of the token x_t is added to get h'_t .

And for the balancing we have a sequence-wise (not batch-wise) auxiliary balancing loss

$$\mathcal{L}_{\text{aux_bal_seq}} = \frac{1}{B} \cdot \frac{N}{k T_{\text{seq}}^2} \sum_{j=1}^{N \cdot B} (P_j f_j)$$

$$= \frac{N}{k T_{\text{seq}}^2 B} \sum_{j=1}^{N \cdot B} (P_j f_j),$$

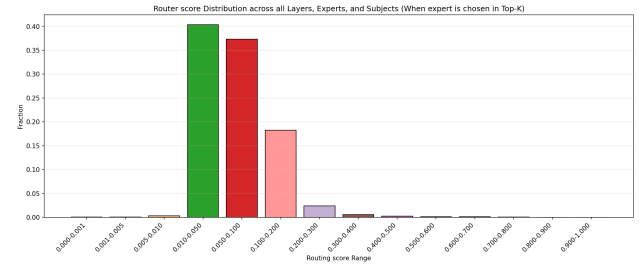
$$f_j = \sum_{x \in T_{\text{seq}}} \text{routing_map}(x, j), \quad f_j \in \mathbb{N}_0^{N \cdot B}$$

$$P_j = \sum_{x \in T_{\text{seq}}} \text{probs}(x, j), \quad P_j \in \mathbb{R}^{N \cdot B}$$

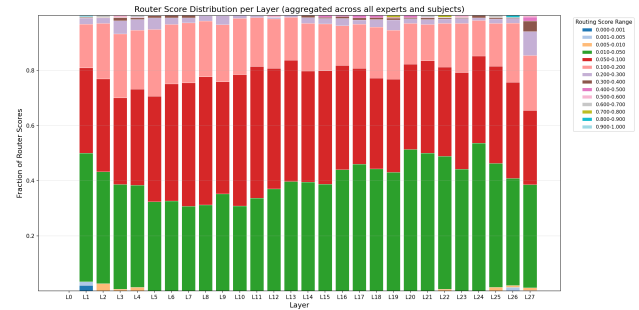
where B is the batch size, T are the number of tokens in a sequence, N the number of experts and K is the size for top-k chosen experts for a token.

3.1.1. ROUTER DISTRIBUTION FOR ALL TOKENS · TOP-K TOTAL TOKENS

For now we will show for this model only the AllenAI ARC eval dataset router distributions, but you can see all of them in A.



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 1. Router distributions for DeepSeek-MoE on AllenAI ARC. We can see that the early middle layers normally assign higher router score values to the experts that the rest of the layers.

Figure 1(a) demonstrates that applying a threshold of $p = 0.05$ would deactivate experts responsible for approximately

40% of Tokens · Top-K operations. And Layer-wise analysis (Figure 1(b)) reveals disproportionate impact on early and penultimate layers, where the deactivation rates exceed 50%.

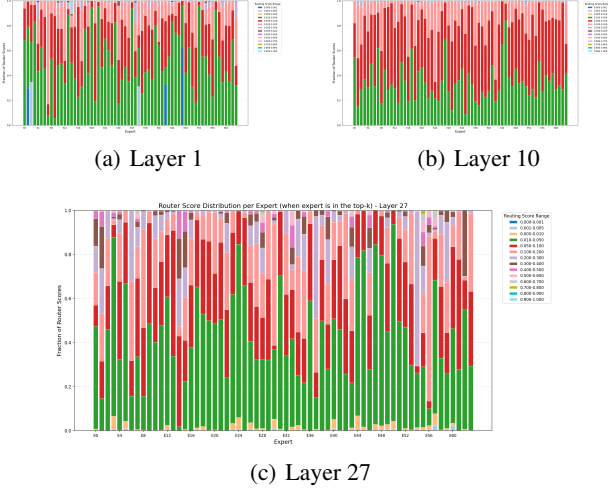


Figure 2. Per-expert router distributions for DeepSeek-MoE on AllenAI ARC at selected layers. We can see that in this dataset many experts in the first and last layer would be deactivated the majority of the time if we chose a threshold of $p = 0.05$

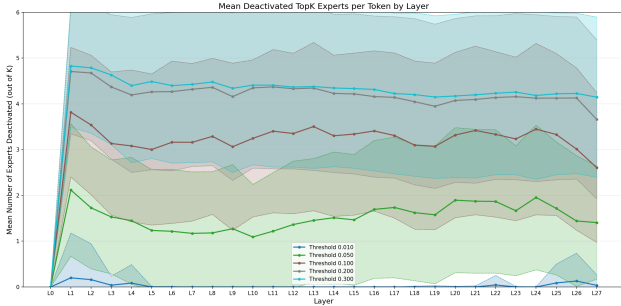


Figure 3. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p

Figure 3 shows that when using a threshold of $p = 0.05$, we would be deactivating on average 1 to 2 experts on the top-k chosen experts for a token t (depending on the layer) and sometimes we would deactivate 3 to 4 experts or 0 experts. I want to put emphasis on this because sometimes we will be deactivating more than half the experts, but we have to remember that we have also our special 2 shared experts and maybe this also demonstrates how this shared experts do majority of the work on very simple tokens and then it needs the top-k experts to work with more special tokens 15.

3.2. OLMoE

First, OLMoE has a Top-k = 8, $N = 64$ experts, and 16 layers in total <https://huggingface.co/allenai/OLMoE-1B-7B-0125/blob/main/config.json>.

For OLMoE (Muennighoff et al., 2025), the MoE architecture and balancing loss that was used is:

$$\begin{aligned} h'_t &= x_t + o_t \\ o_t &= \sum_{i=1}^N g_{i,t} \text{FFN}_i(x_t), \\ g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases} \\ s_{i,t} &= \text{Softmax}_i(W_i x_t) \end{aligned}$$

where N is the number of experts, K is the number of top-k chosen experts for a token t , x_t is the token t , and h'_t is the new hidden state for token t after we sum the residual to the weighted sum output.

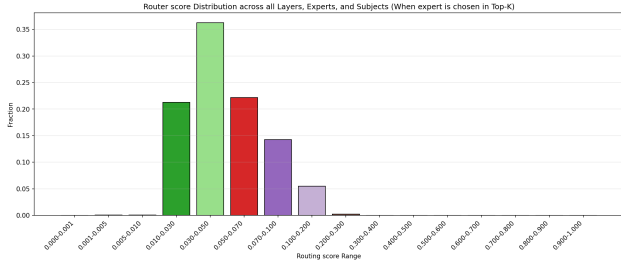
$$\begin{aligned} L_{LB} &= N \cdot \sum_{i=1}^N f_i \cdot P_i \\ f_i &= \frac{1}{KT} \sum_{x \in T} \text{routing_map}(x, i) \\ P_i &= \frac{1}{T} \sum_{x \in T} \text{probs}(x, i) \end{aligned}$$

where the balancing loss used is a batch-wise balancing loss, $T = \text{micro_batch_size} \cdot \text{sequence_length}$, probs is the probability map that says how much probability token t gives to expert i and the routing map is the indicator function (giving one if an expert is chosen in the top-k for token t).

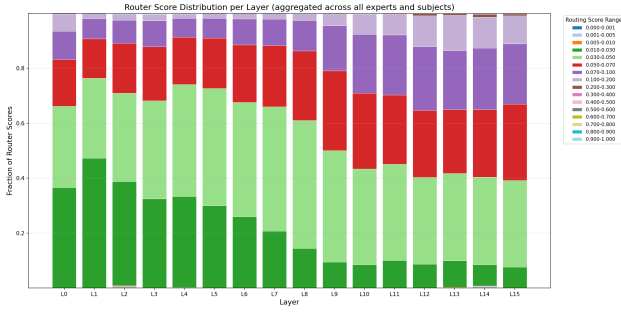
3.2.1. ROUTER DISTRIBUTION FOR ALL TOKENS · TOP-K TOTAL TOKENS

Unlike DeepSeek-MoE, OLMoE doesn't use any shared experts (as AllenAI observed equivalent or degraded performance with shared experts in their experiments) and utilizes batch-wise rather than sequence-wise auxiliary balancing losses (Dai et al., 2024).

For OLMoE we can see that if we use a threshold of $p = 0.03$ we would be deactivating $\approx 20\%$ of the operations and if we use $p = 0.05$ we would be deactivating $\approx 55\%$ of the operations done on the Tokens · Top-K total tokens. And if we look at the layers is only in the first layers where we would have a very big impact for middle and last layers we would be deactivating $< 10\%$ of the operations in this layer for the total Tokens · Top-K total tokens in that layer.



(a) Global router distribution



(b) Per-layer router distribution

Figure 4. Router distributions for OLMoE on MMLU.

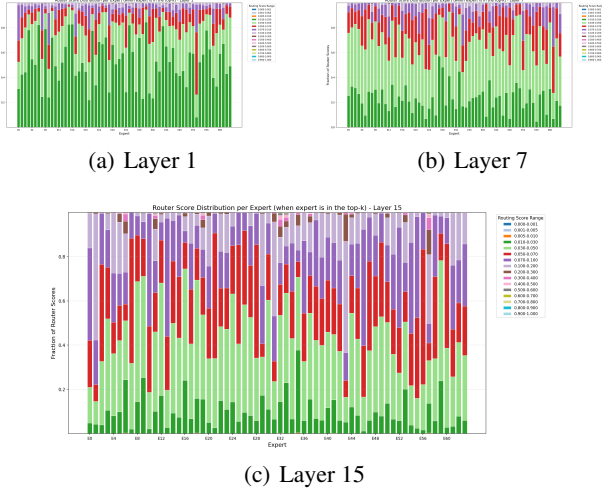
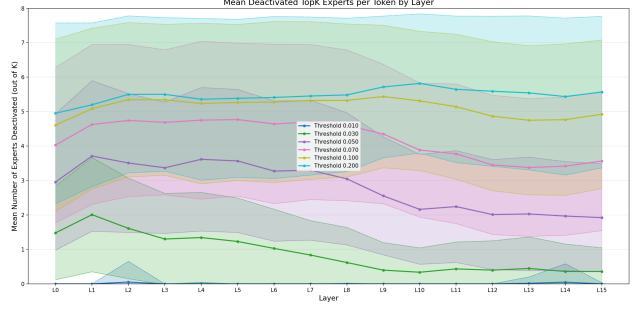


Figure 5. Per-expert router distributions for OLMoE on MMLU at selected layers.


 Figure 6. Deactivated experts by a threshold p in the Top-k for MMLU (Hendrycks et al., 2021b) (Hendrycks et al., 2021a)

We can see from Figure 6 that if we used a threshold of $p = 0.03$ we would be deactivating on average 2 to almost 0 experts in the Top-k for a token t . Thus, for OLMoE we do not observe impressive threshold deactivation results (Figure 13); at performance preserving thresholds ($p \approx 0.30$), we do not deactivate many experts per token. We think that this limitation comes from the absence of shared experts and the use of batch-wise auxiliary balancing losses, which affect router scores more directly. **However, in this case maybe high router scores do not necessarily imply that experts want a token, and they instead learn to output $f : x \rightarrow 0$ for dispreferred tokens even when selected in the top- k .** This report doesn't visualize expert outputs for specific tokens, but we think that such analysis is necessary to fully characterize specialization, as experts may learn to modulate their weighted sum contributions independently of routing weights.

3.3. Trinity

First for Trinity we have; 1 shared expert, Top-k = 8, $N = 128$ routable experts, 56 layers with the first 2 being dense layers and a router scale of $\lambda = 2.826$ (<https://huggingface.co/arcee-ai/Trinity-Nano-Base/blob/main/config.json>).

For Trinity the routing method and balancing method that was used is;

$$\begin{aligned}
 h'_t &= x_t + o_t \\
 o_t &= \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(x_t) + \sum_{i=1}^{N_r} \lambda g_{i,t} \text{FFN}_i^{(r)}(x_t) \\
 g_{i,t} &= \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}} \\
 g'_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j \mid 1 \leq j \leq N_r\}, K), \\ 0, & \text{otherwise,} \end{cases} \\
 s_{i,t} &= \text{Sigmoid}(W_i^g x_t),
 \end{aligned}$$

Here N_r are the routable experts, N_s the shared experts, W_g the routers weight, K is the top-k chosen experts for a token, and λ is the routers scale. For the router scale I can't give an explanation of what it is supposed to do, as this value was introduced in Deepseek-V3 <https://huggingface.co/deepseek-ai/DeepSeek-V3/blob/main/config.json> but its theoretical deviation wasn't ever mentioned in (DeepSeek-AI et al., 2025). This λ is just a scalar that from what I understand is not even trained, it just a value you choose. The other strange thing is that before the weighted sum we do a renormalization, so that the values that will be used in the weighted sum are a valid probability distribution and we can better give more weight to one expert over the other, but that is the thing that this is the same as the other models, that is why I don't understand what is the point of this router scale λ value.

$$\bar{c}_i = \frac{1}{N_r} \sum_{i=1}^{N_r} c_i$$

$$e_i = c_i - \bar{c}_i$$

$$\Delta b_i = b_i + u \cdot \text{sign}(e_i)$$

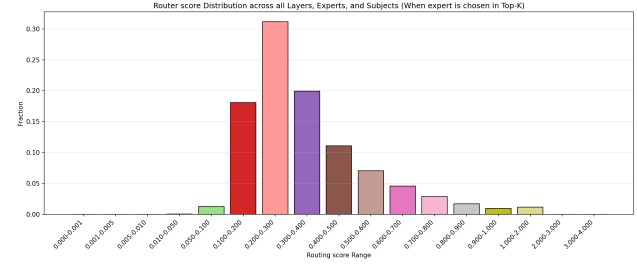
$$b_i = b_i + \Delta b_i$$

$$b_i = b_i - \frac{1}{N_r} \sum_{j=1}^{N_r} b_j$$

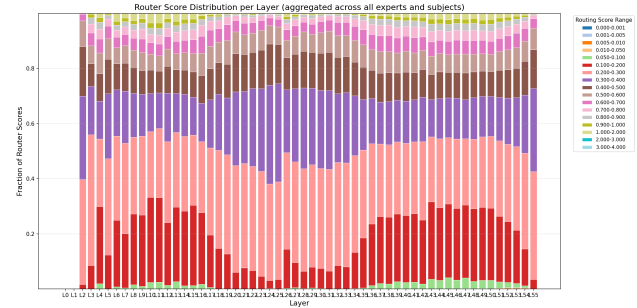
$$\begin{aligned}
 \mathcal{L}_{\text{aux_bal_seq}} &= \frac{1}{B} \cdot \frac{N_r}{k T_{\text{seq}}^2} \sum_{j=1}^{N_r \cdot B} (P_j f_j) \\
 &= \frac{N_r}{k T_{\text{seq}}^2 B} \sum_{j=1}^{N_r \cdot B} (P_j f_j), \\
 f_j &= \sum_{x \in T_{\text{seq}}} \text{routing_map}(x, j), \quad f_j \in \mathbb{N}_0^{N_r \cdot B} \\
 P_j &= \sum_{x \in T_{\text{seq}}} \text{probs}(x, j), \quad P_j \in \mathbb{R}^{N_r \cdot B}
 \end{aligned}$$

For the balancing method, they use the auxiliary loss free method (which optimizes at the global batch wise level) (Wang et al., 2024a) in conjunction with the sequence-wise auxiliary balancing loss as it was done in (DeepSeek-AI et al., 2025). Here B is the batch size, T is the sequence length, N_r is the routable experts, routing_map is the indicator function and probs is the probability map map of token t choosing expert i (as we need a valid probability distribution to use the auxiliary balancing loss).

3.3.1. ROUTER DISTRIBUTION FOR ALL TOKENS · TOP-K TOTAL TOKENS



(a) Global router distribution



(b) Per-layer router distribution

Figure 7. Router distributions for Trinity on MMLU (Hendrycks et al., 2021b) (Hendrycks et al., 2021a).

For trinity 47(a) if we used a threshold of $p = 0.30$ we would deactivate $\approx 50\%$ of the operations in the total Tokens · Top-k tokens. And if we see this threshold per layer 47(b), we would deactivate $> 50\%$ of operations the total Tokens · Top-k tokens.

At $p = 0.30$, Trinity deactivates on average 2 experts per token (standard deviation permitting 0 to 5 deactivated experts per token), occasionally eliminating 60% of routed experts in the first layers. Remarkably, performance remains stable despite such aggressive pruning (Figure 14). The shared expert could maybe provide a critical stabilization, as also evidenced by near-zero utilization of many routed experts in early layers suing this threshold this threshold (Figure 47(b)).

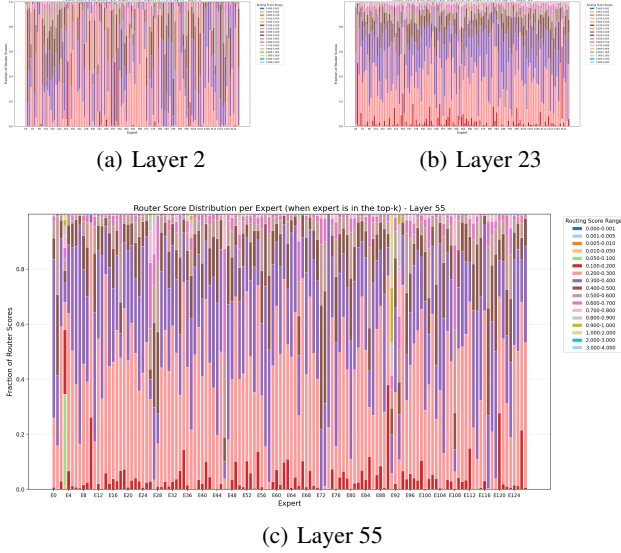


Figure 8. Per-expert router distributions for Trinity on MMLU at selected layers.

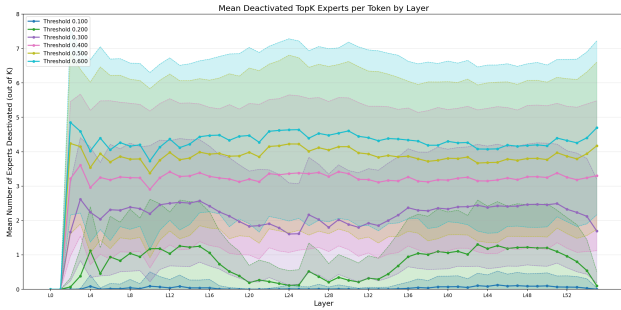


Figure 9. Number of deactivated experts by a threshold p per token in the Top-k chosen expert for token t

3.4. GPT-OSS

First GPT-Oss 20B uses $N = 32$ experts, Top-k = 4 and 24 hidden layers in total

The routing and balancing method used is (https://github.com/huggingface/transformers/blob/v4.56.2/src/transformers/models/gpt_oss/modeling_gpt_oss.py):

$$h'_t = x_t + o_t$$

$$o_t = \sum_{i=1}^N g_{i,t} \text{FFN}_i(x_t),$$

$$l_{i,t} = W_i^g x_t$$

$$g_{i,t} = \begin{cases} l_{i,t}, & l_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases}$$

$$s_{i,t} = \frac{g_{i,t}}{\sum_{j=1}^N g_{j,t}}$$

Where W_g is the routers weight, x_t is the t token, and h'_t is the new hidden state after we add the residual to the weight sum output o_t . For GPT-OSS we do a softmax after Top-k instead of; doing Softmax, then Top-k, and then not renormalizing like in OLMoE.

$$L_{LB} = N \cdot \sum_{i=1}^N f_i \cdot P_i f_i = \frac{1}{T} \sum_{x \in T} \text{routing_map}(x, i)$$

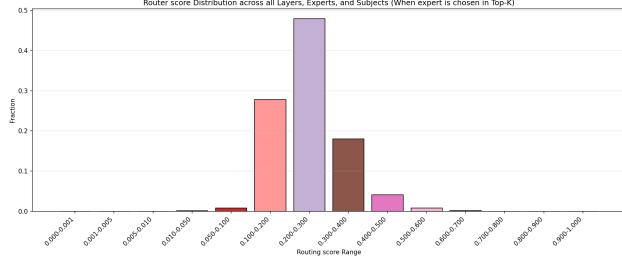
$$P_i = \frac{1}{T} \sum_{x \in T} \text{probs}(x, i)$$

For the balancing loss I can't say for sure if the one I'm mentioning is the correct one, because it is not mentioned in the paper and the loss they are using in the huggingface code https://github.com/huggingface/transformers/blob/cd74917ffc3e8f84e4a886052c5ab32b7ac623cc/src/transformers/models/gpt_oss/modeling_gpt_oss.py#L572C8-L582C10 has an error, where f_i is not being normalized by K , not doing this makes it so that $\sum_{i=1}^N f_i = K$ instead of 1 and because of this $f_i = \frac{K}{N}$ is trying to approximate another distribution compared to $P_i = \frac{1}{N} (\sum_{i=1}^N P_i = 1)$ that is trying to do a uniform distribution.

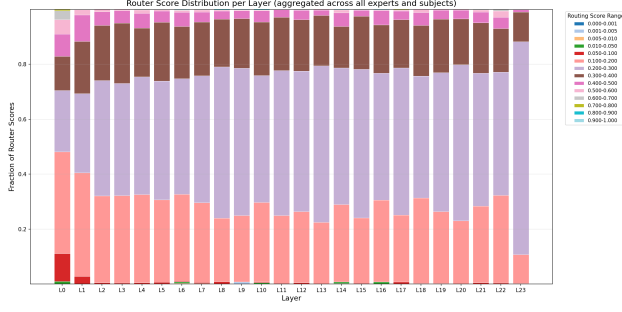
3.4.1. ROUTER DISTRIBUTION FOR ALL TOKENS · TOP-K TOTAL TOKENS

For GPT-OSS if we used a threshold of $p = 0.2$ we would be deactivating $\approx 30\%$ of the operations in the total layers ·

MoE Router distribution exploration



(a) Global router distribution



(b) Per-layer router distribution. The first layers likes to give more lower and higher values compared to the next layers

Figure 10. Router distributions for GPT-OSS on MMLU.

Tokens · Top-k tokens 62(a). And If we look at each layer 62(b), if we used $p = 0.2$ we would deactivate around 20% to 30% of the operations in the total Tokens · Top-k.

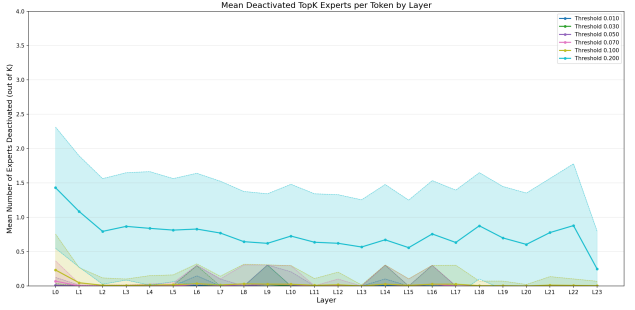
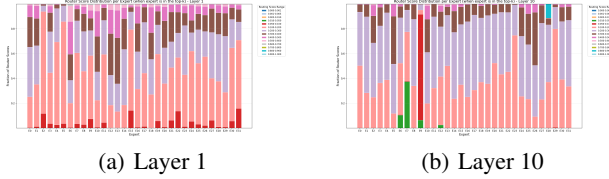
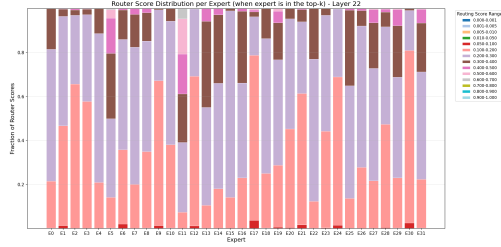


Figure 12. Number of deactivated experts in the top-k of a token for GPT-OSS in MMLU



(a) Layer 1

(b) Layer 10



(c) Layer 22

Figure 11. Per-expert router distributions for GPT-OSS on MMLU at selected layers.

And using a threshold of $p = 0.2$ for GPT-OSS 12 we would only deactivate on average around one expert out of the Top-4 experts for a token in MMLU.

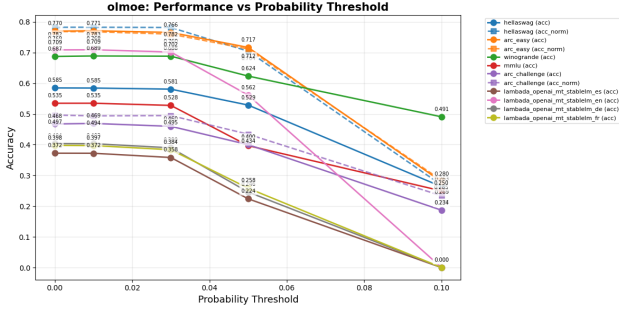


Figure 13. OLMoE 2025 performance vs. score threshold

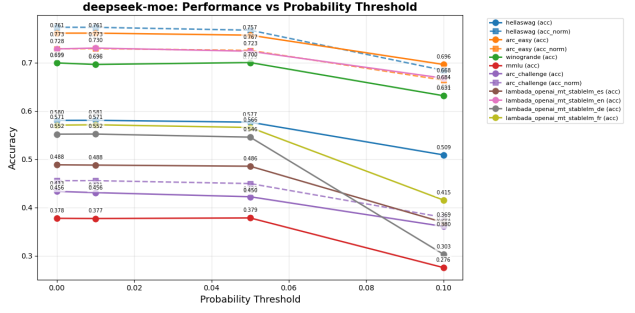


Figure 15. DeepSeek MoE performance vs. score threshold

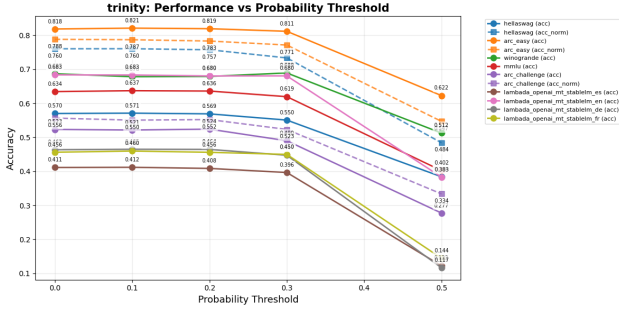


Figure 14. Trinity nano performance vs. score threshold

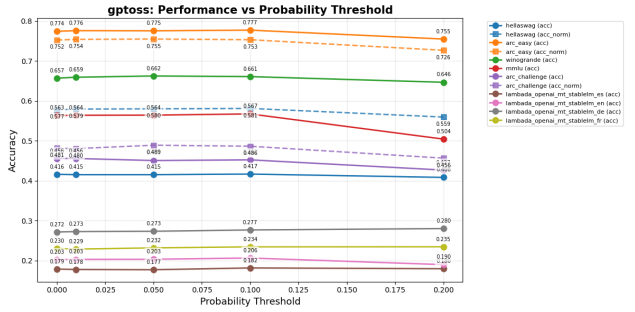


Figure 16. GPT OSS performance vs. score threshold

4. Results

For the weighted sum $\sum_{i=1}^N g_i(x_t)FFN_i(x_t)$, we do $g(x_t) = 0$ if $g(x_t) < p$ for the probability (or score) values of the top-k experts selected for token t .

The idea here is that maybe the router learns to give a low score to experts that are even in the top-k chosen experts for a model, so as to also help specialize the experts in the model so they don't get signal from tokens they don't care about, so based on this idea that maybe it is possible to not do computation for some experts without having to train a model that is forced to this from scratch we do this experiment.

Also, I can't say for sure the results for GPT-OSS are correct as I never use the template when doing the exploration of the routers distribution or its evaluations, this is the only model where I'm not using the base model as GPT-OSS doesn't have one.

It seems that models with shared experts (Trinity and DeepSeek-MoE) demonstrate robust performance despite deactivating up to 50% of routed expert operations in the Tokens · Top-K across all layers (Figures 14, 15). In contrast, OLMoE exhibits sensitivity to threshold-based deactivation (Figure 13). We hypothesize that this divergence comes from architectural differences; the absence of shared ex-

perts and the use of auxiliary balancing losses may result in router scores that correlate less with expert utility in the weighted sum. Consequently, zeroing expert based on router scores alone may be insufficient for OLMoE, future work should also investigate output magnitudes rather than router probabilities alone.

We also note that OLMoE employs a micro-batch size of 64 for its batch-wise balancing loss (Muennighoff et al., 2025). The impact of this hyperparameter on expert specialization remains under-explored. And another model that could be interesting to explore one would be Qwen3-MoE, as this model seems to be the only one to have the notable exception in using a Global-batch wise auxiliary balancing loss instead of a sequence-wise or micro-batch-wise (Yang et al., 2025).

5. Conclusion

It seems interesting that even if we have a very high router score, like in the trinity case because of the router scaling, even if we deactivate many of the experts the performance doesn't drop. It's also interesting seeing that OLMoE doesn't get very good results if we deactivate many of the experts.

The first thing I thought based on this results is that it does seem that DeepSeek was right and shared experts can help

a lot with a the model and make experts more specialized. But the second thing I thought is that maybe we are putting too much emphasis to the router when studying the idea of expert "specialization", and what if maybe the experts are also the ones that are learning what tokens they want to work with, meaning that even if the router want's to choose an expert for the top-k and the router score can maybe be somewhat high because of the balancing loss that was used, maybe the expert still decides to give a value close to zero $f : x \rightarrow 0$ so as to not be counted in the weighted sum.

We have to remember that balancing loss affects directly the score of the router for the experts to be able to represent the uniform distribution that is being desired (sequence-wise, micro-batch wise or global-batch wise). Compare this to the auxiliary-loss free method with the bias, this one doesn't modify the scores of the router directly, it just adds a value to the output of the gate of the router before doing the top-k (and this value is not used in the weighted sum of the experts). So maybe the auxiliary balancing loss doesn't affect the MoE's performance, but it can maybe affect what the router score represents, meaning they will no longer represent how much does a token an expert care about each other, it will now represent in part how much does a token care about an expert but also it just represents the uniform distribution we want to achieve to be able to not have some experts always doing all the work and the others not being utilized. And in this case now the experts have to learn a representation somewhat different of the router, meaning even if the router cares about the expert, the expert doesn't and just outputs a very small value so as to not be considered in the weighted sum.

So maybe when we study MoE specialization we should maybe also peek at the expert's raw router score for that token, because maybe the expert doesn't want to work with that token, but because we need to have K experts for a given token it is still chosen by the router and the router maybe is just learning more about how to distribute the work based on the balancing loss we are applying to it and thats why it still outputs a somewhat big score for that expert in the weighted sum.

6. Future Work

- It would be interesting to be able to explore other models router distribution, like Qwen3-MoE as this one uses a global batch-wise auxiliary balancing loss or bigger MoE models that have a bigger number of routable experts, and see their performance when deactivating the experts based on the p threshold
- It would also be interesting to explore the distribution of the expert outputs and compare them to the router scores that are being given to a token t , so as to get an

idea if router score and expert outputs work together or does the expert tries to do its own work in the weighted sum and try to deactivate itself if it doesn't care about a specific token

References

- Winogrande: An adversarial winograd schema challenge at scale. 2019.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.
- Dai, D., Deng, C., Zhao, C., Xu, R. X., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., Xie, Z., Li, Y. K., Huang, P., Luo, F., Ruan, C., Sui, Z., and Liang, W. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models, 2024. URL <https://arxiv.org/abs/2401.06066>.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W. L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhu, Y. X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z.,

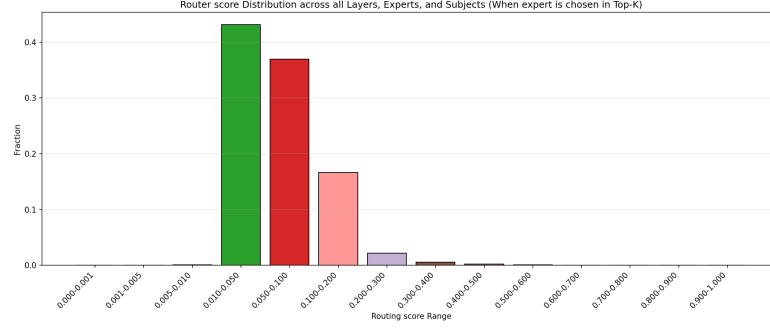
- and Pan, Z. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h, A., Li, H., McDonnell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- Hendrycks, D., Burns, C., Basart, S., Critch, A., Li, J., Song, D., and Steinhardt, J. Aligning ai with shared human values. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021a.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021b.
- Jin, P., Zhu, B., Yuan, L., and Yan, S. Moe++: Accelerating mixture-of-experts methods with zero-computation experts, 2024. URL <https://arxiv.org/abs/2410.07348>.
- Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., Gu, Y., Arora, S., Bhagia, A., Schwenk, D., Wadden, D., Wettig, A., Hui, B., Dettmers, T., Kiela, D., Farhadi, A., Smith, N. A., Koh, P. W., Singh, A., and Hajishirzi, H. Olmoe: Open mixture-of-experts language models, 2025. URL <https://arxiv.org/abs/2409.02060>.
- Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q. N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The lambda dataset, Aug 2016.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019.
- Singh, V., Krauss, L., Jaghouar, S., Sirovatka, M., Goddard, C., Obeid, F., Ong, J. M., Straube, J., Fern, Harley, A., Stewart, C., Kealty, C., Panahi, M., Kirsten, S., Deshpande, A., Vij, A., Bresnu, A., Veldurthi, P., Ravishankar, R., Bishnoi, H., McQuade, M., Hagemann, J., and Atkins, L. Arcee trinity large technical report, 2026. URL <https://github.com/arcee-ai/trinity-large-tech-report>.
- Team, M. L., Bayan, Li, B., Lei, B., Wang, B., Rong, B., Wang, C., Zhang, C., Gao, C., Zhang, C., Sun, C., Han, C., Xi, C., Zhang, C., Peng, C., Qin, C., Zhang, C., Chen, C., Wang, C., Ma, D., Pan, D., Bu, D., Zhao, D., Kong, D., Liu, D., Huo, F., Li, F., Zhang, F., Dong, G., Liu, G., Xu, G., Li, G., Tan, G., Lin, G., Jing, H., Fu, H., Yan, H., Wen, H., Zhao, H., Liu, H., Shi, H., Hao, H., Tang, H., Lv, H., Su, H., Li, J., Liu, J., Li, J., Yang, J., Wang, J., Yang, J., Tan, J., Sun, J., Zhang, J., Fu, J., Yang, J., Hu, J., Qin, J., Wang, J., He, J., Kuang, J., Mei, J., Liang, K., He, K., Zhang, K., Wang, K., He, K., Gao, L., Shi, L., Ma, L., Qiu, L., Kong, L., Si, L., Lyu, L., Guo, L., Yang, L., Yan, L., Xia, M., Gao, M., Zhang, M., Zhou, M., Shen, M., Tuo, M., Zhu, M., Li, P., Pei, P., Zhao, P., Jia, P., Sun, P., Gu, Q., Li, Q., Li, Q., Huang, Q., Duan, Q., Meng, R., Weng, R., Shao, R., Li, R., Wu, S., Liang, S., Wang, S., Dang, S., Fang, T., Li, T., Chen, T., Bai, T., Zhou, T., Xie, T., He, W., Huang, W., Liu, W., Shi, W., Wang, W., Wu, W., Zhao, W., Zan, W., Shi, W., Nan, X., Su, X., Li, X., Mei, X., Ji, X., Xi, X., Huang, X., Li, X., Fu, X., Liu, X., Wei, X., Cai, X., Chen, X., Liu, X., Li, X., Shi, X., Li, X., Wang, X., Chen, X., Hu, X., Miao, X., He, X., Zhang, X., Hao, X., Cao, X., Cai, X., Yang, X., Feng, Y., Bai, Y., Chen, Y., Yang, Y., Huo, Y., Sun, Y., Lu, Y., Zhang, Y., Zang, Y., Zhai, Y., Li, Y., Yin, Y., Lv, Y., Zhou, Y., Yang, Y., Xie, Y., Sun, Y., Zheng, Y., Wei, Y., Qian, Y., Liang, Y., Tai, Y., Zhao, Y., Yu, Z., Zhang, Z., Yang, Z., Zhang, Z., Xia, Z., Zou, Z., Zeng, Z., Su, Z., Chen, Z., Zhang, Z., Wang, Z., Jiang, Z., Zhao, Z., Wang, Z., and Su, Z. Longcat-flash technical report, 2025. URL <https://arxiv.org/abs/2509.01322>.
- Wang, L., Gao, H., Zhao, C., Sun, X., and Dai, D. Auxiliary-loss-free load balancing strategy for mixture-of-experts, 2024a. URL <https://arxiv.org/abs/2408.15664>.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024b. URL <https://arxiv.org/abs/2406.01574>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

Zeng, Z., Miao, Y., Gao, H., Zhang, H., and Deng, Z.
Adamoe: Token-adaptive routing with null experts for
mixture-of-experts language models, 2024. URL <https://arxiv.org/abs/2406.13233>.

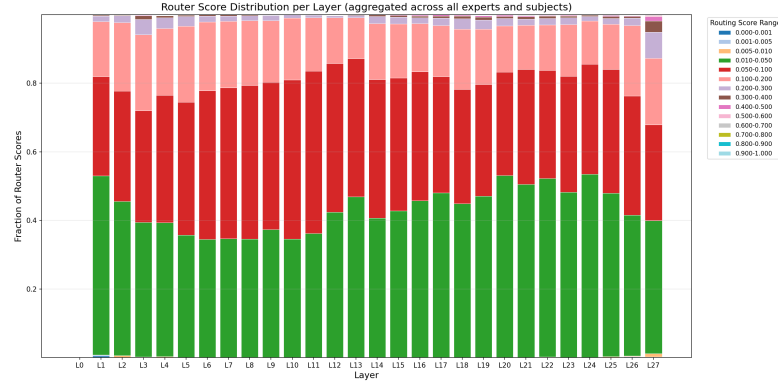
A. Router distributions for all eval datasets

A.1. Deepseek-MoE

A.1.1. MMLU



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 17. Router distributions for DeepSeek-MoE on MMLU.

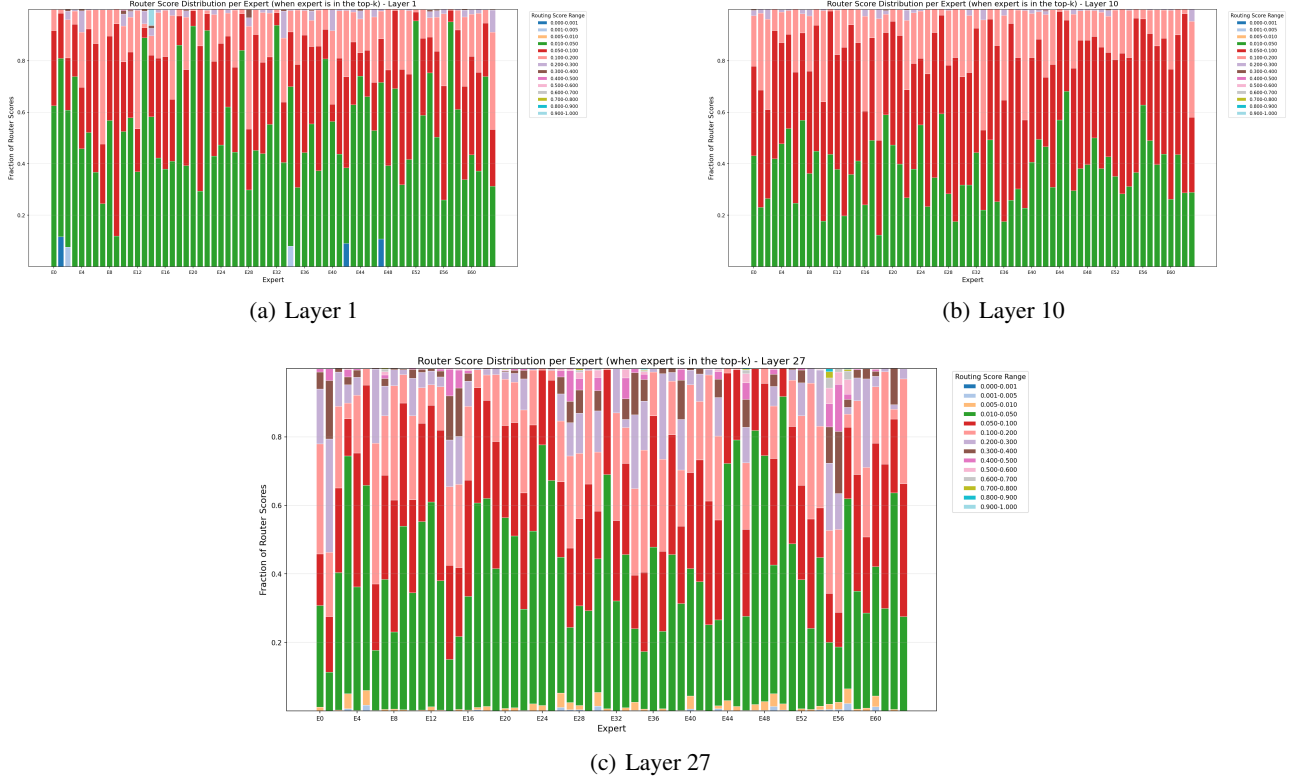


Figure 18. Per-expert router distributions for DeepSeek-MoE on MMLU at selected layers.

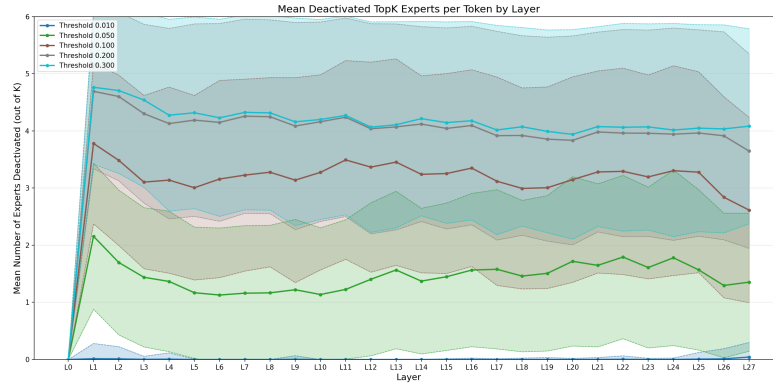
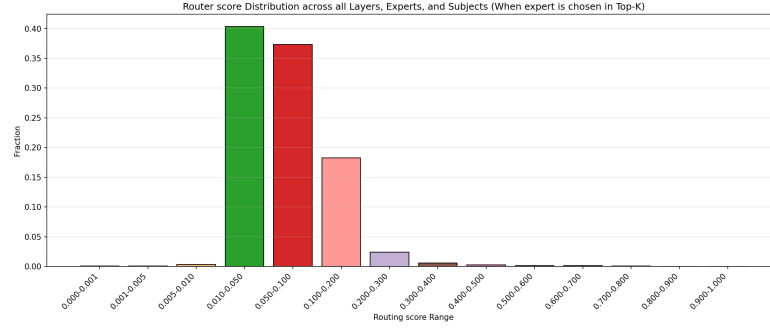
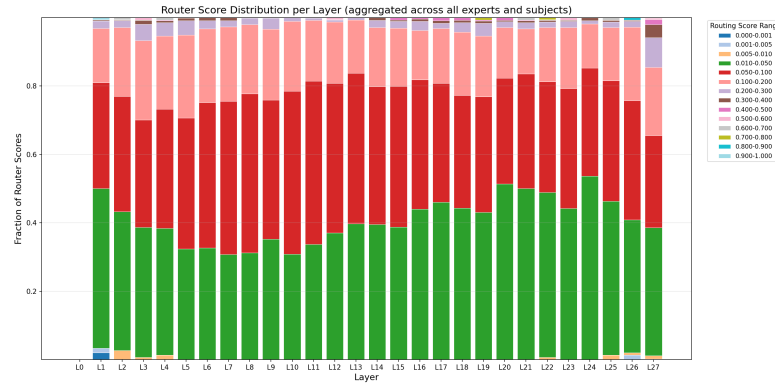


Figure 19. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU.

A.1.2. ALLENAI ARC



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 20. Router distributions for DeepSeek-MoE on AllenAI ARC. We can see that the early middle layers normally assign higher router score values to the experts that the rest of the layers.

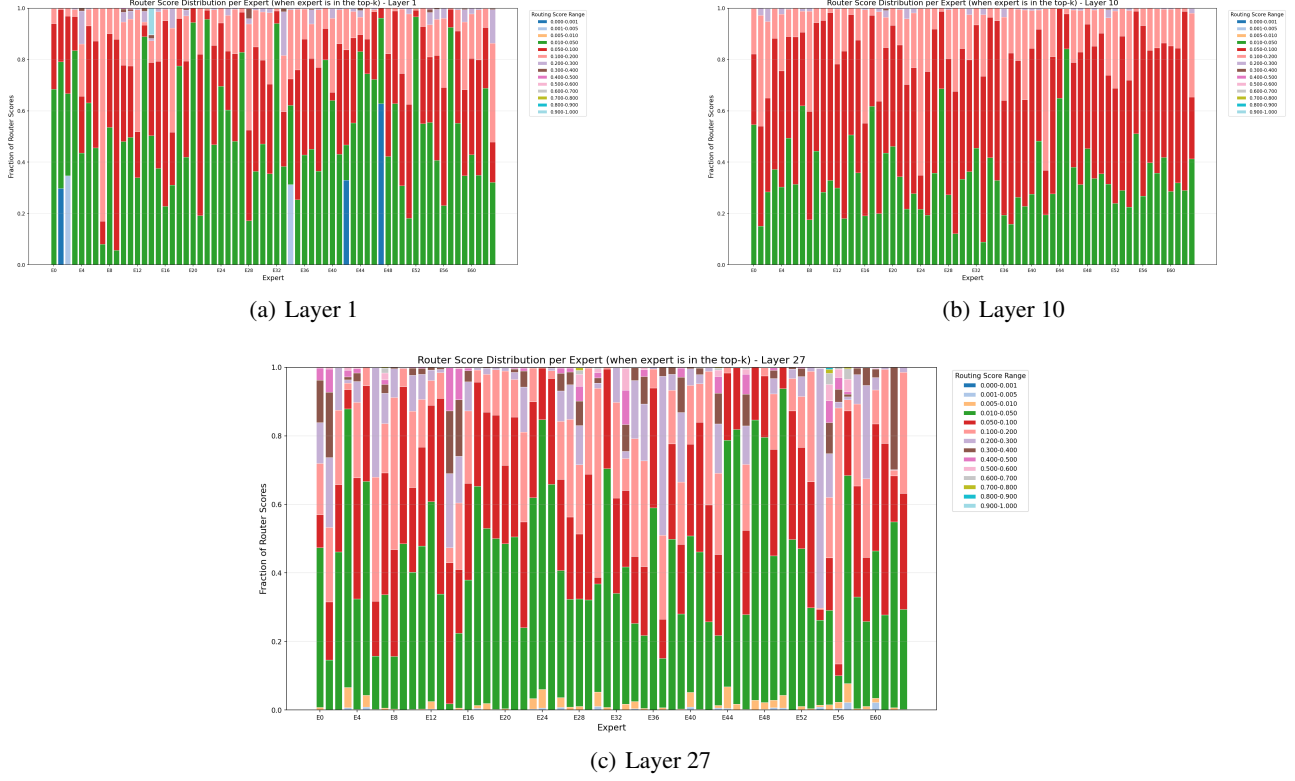


Figure 21. Per-expert router distributions for DeepSeek-MoE on AllenAI ARC at selected layers. We can see that in this dataset many experts in the first and last layer would be deactivated the majority of the time if we chose a threshold of $p = 0.05$

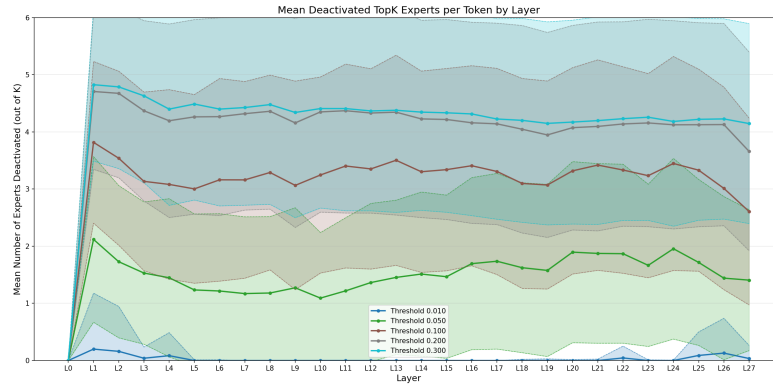
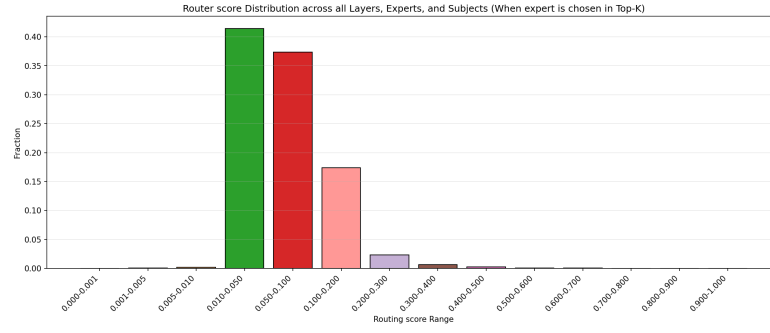
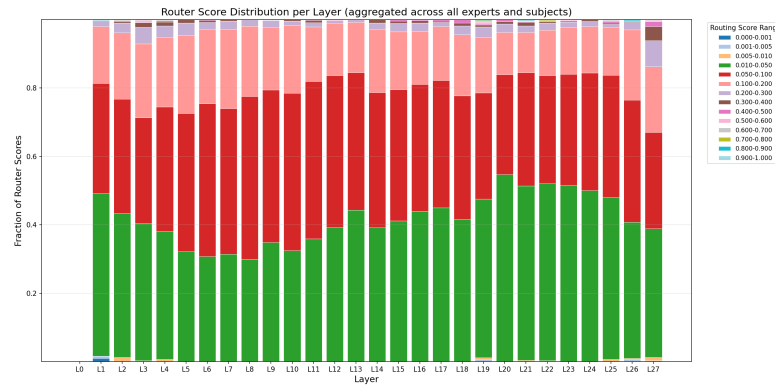


Figure 22. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for AllenAI ARC.

A.1.3. HELLASWAG



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 23. Router distributions for DeepSeek-MoE on Hellaswag.

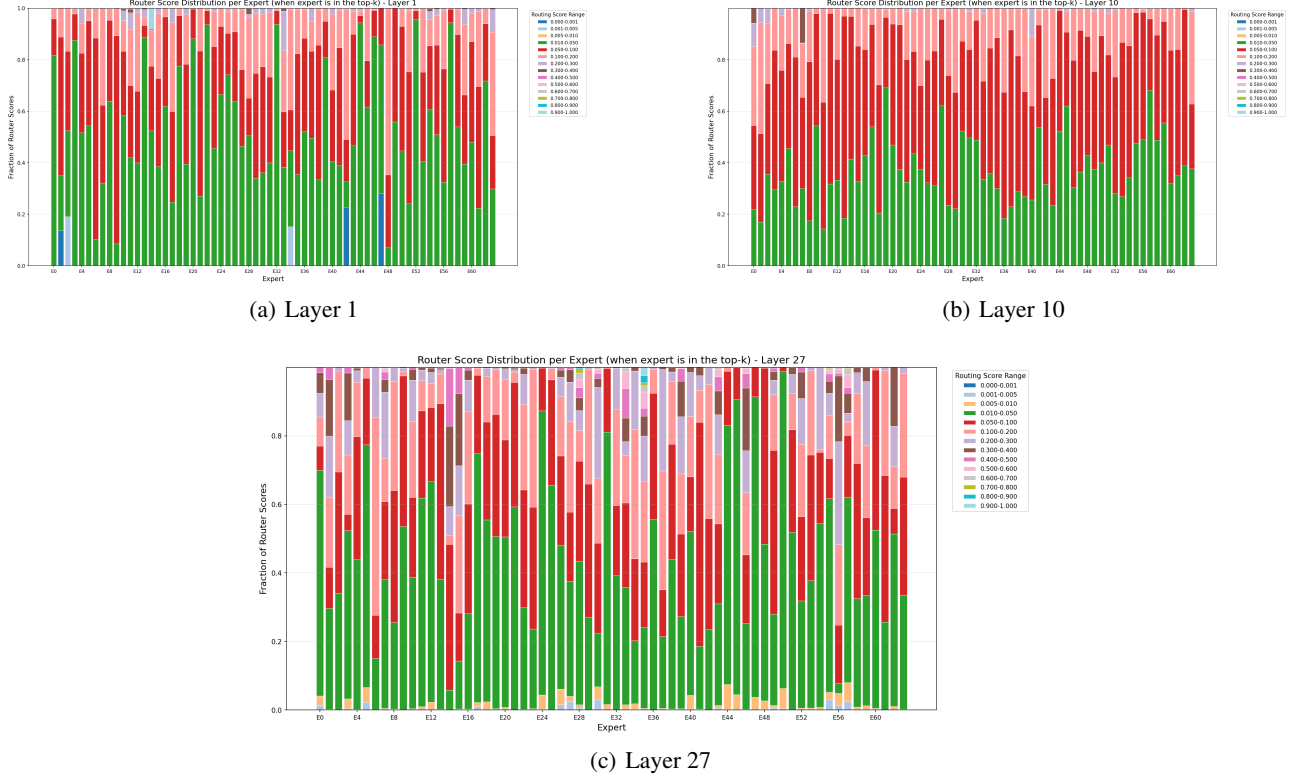


Figure 24. Per-expert router distributions for DeepSeek-MoE on Hellaswag at selected layers.

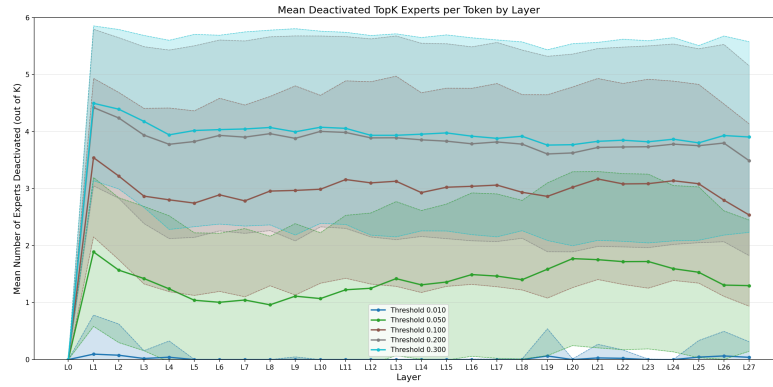
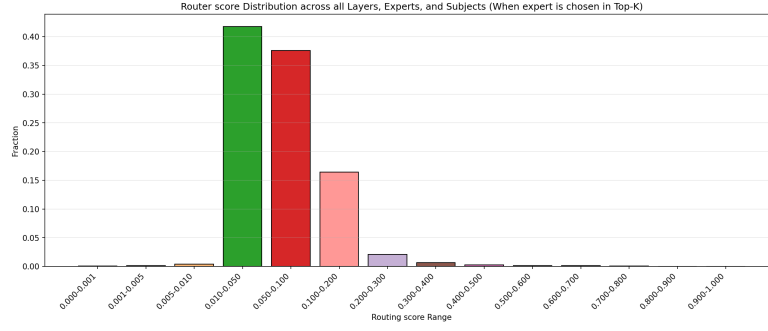
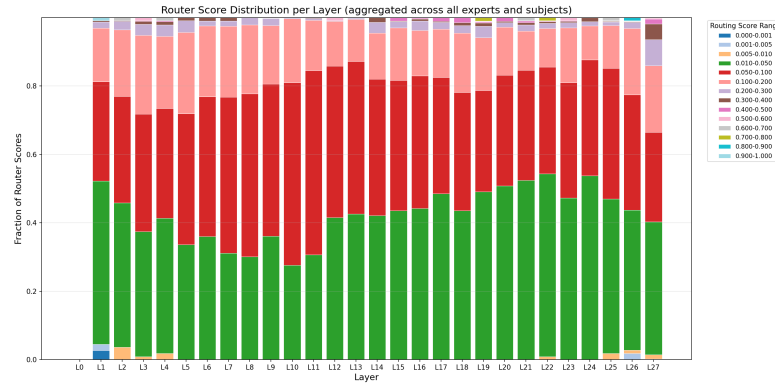


Figure 25. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Hellaswag.

A.1.4. WINOGRANDE



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 26. Router distributions for DeepSeek-MoE on Winogrande.

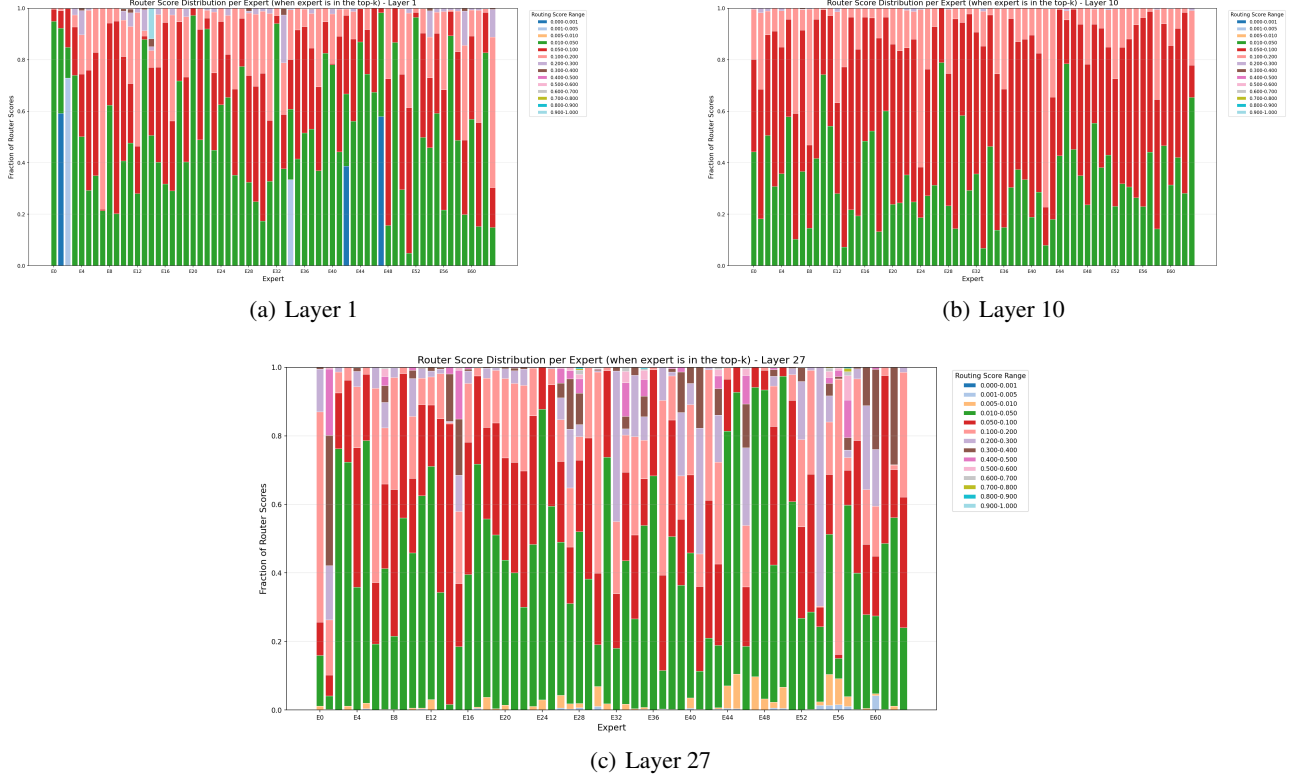


Figure 27. Per-expert router distributions for DeepSeek-MoE on Winogrande at selected layers.

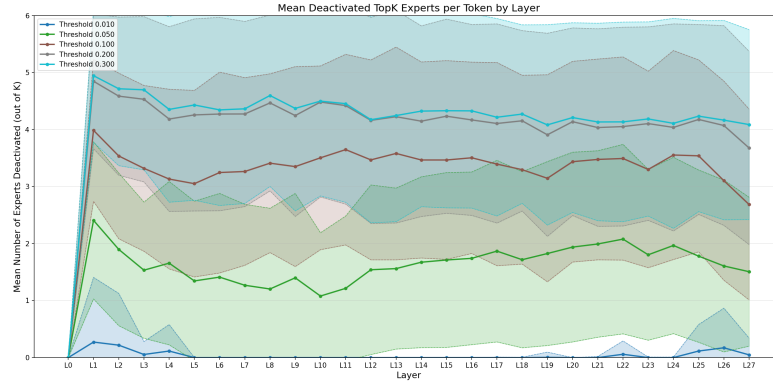
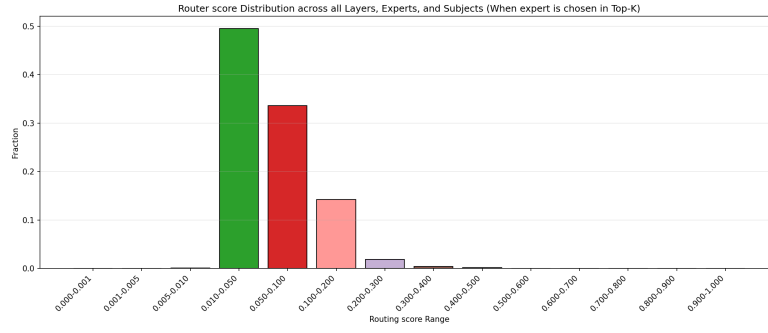


Figure 28. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Winogrande.

A.1.5. MMLU PRO



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 29. Router distributions for DeepSeek-MoE on MMLU Pro.

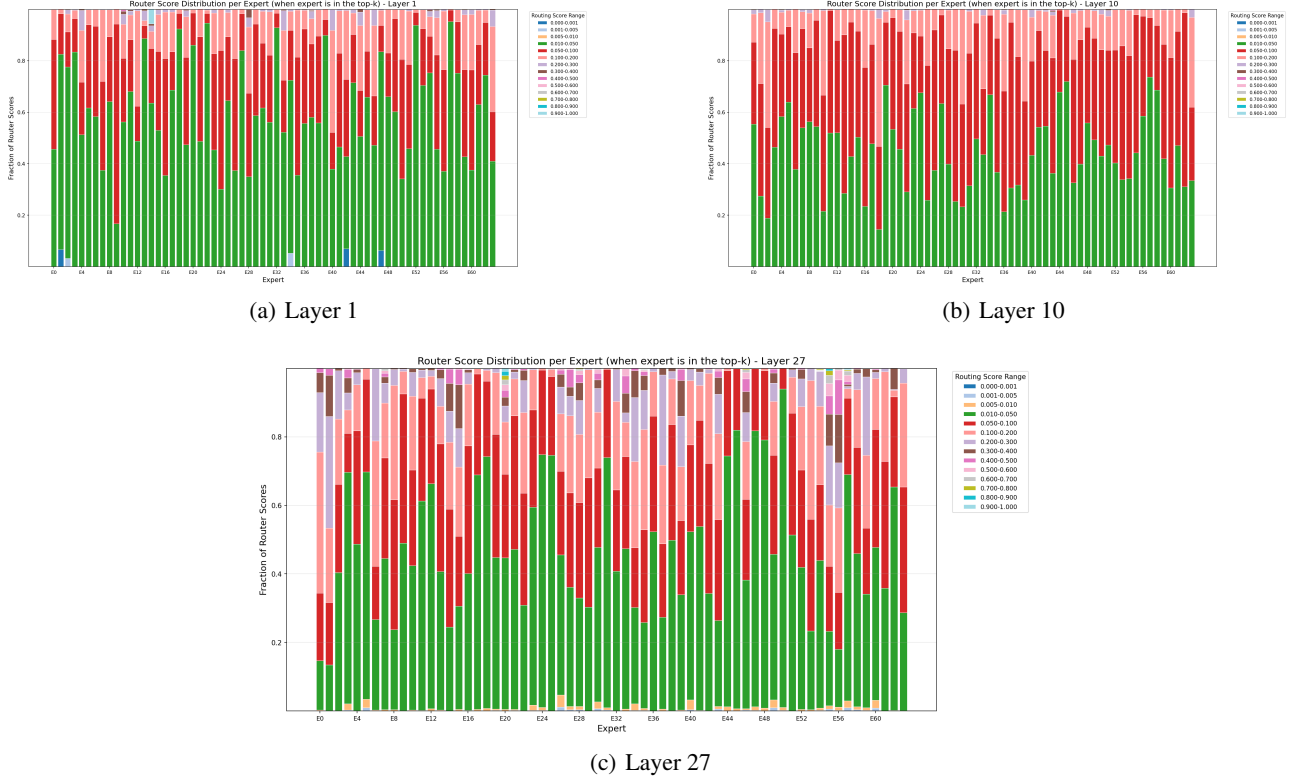


Figure 30. Per-expert router distributions for DeepSeek-MoE on MMLU Pro at selected layers.

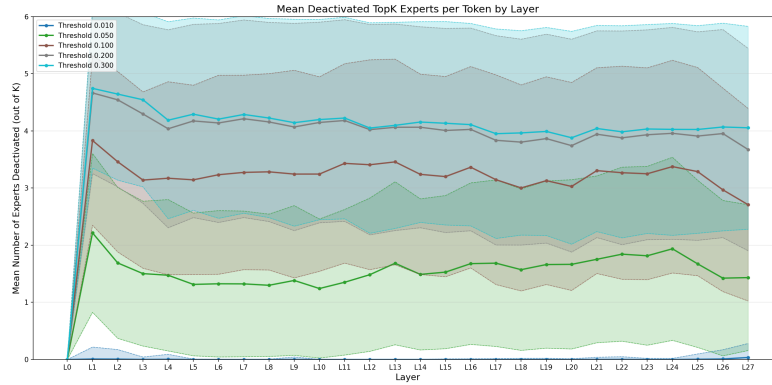
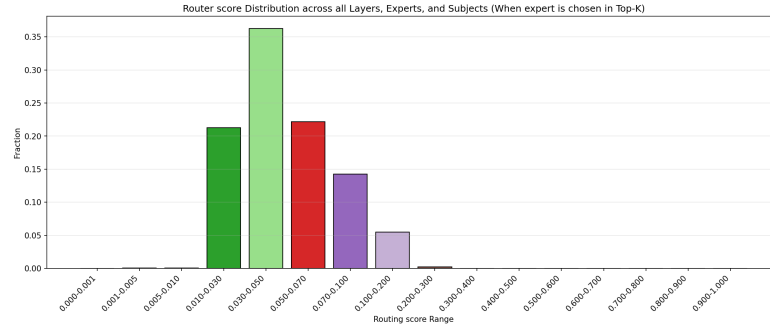


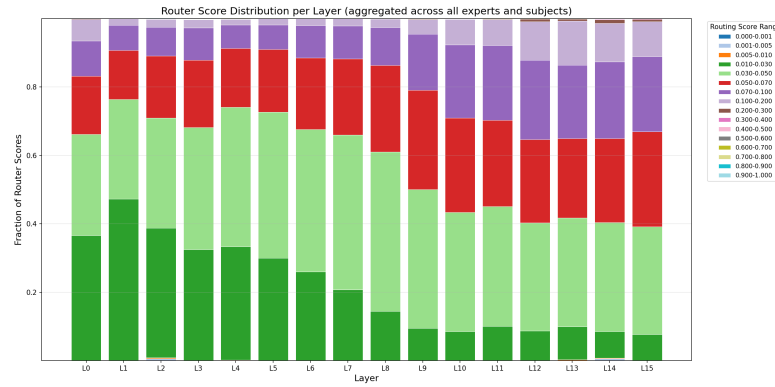
Figure 31. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU Pro.

A.2. OLMoE

A.2.1. MMLU



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 32. Router distributions for OLMoE on MMLU.

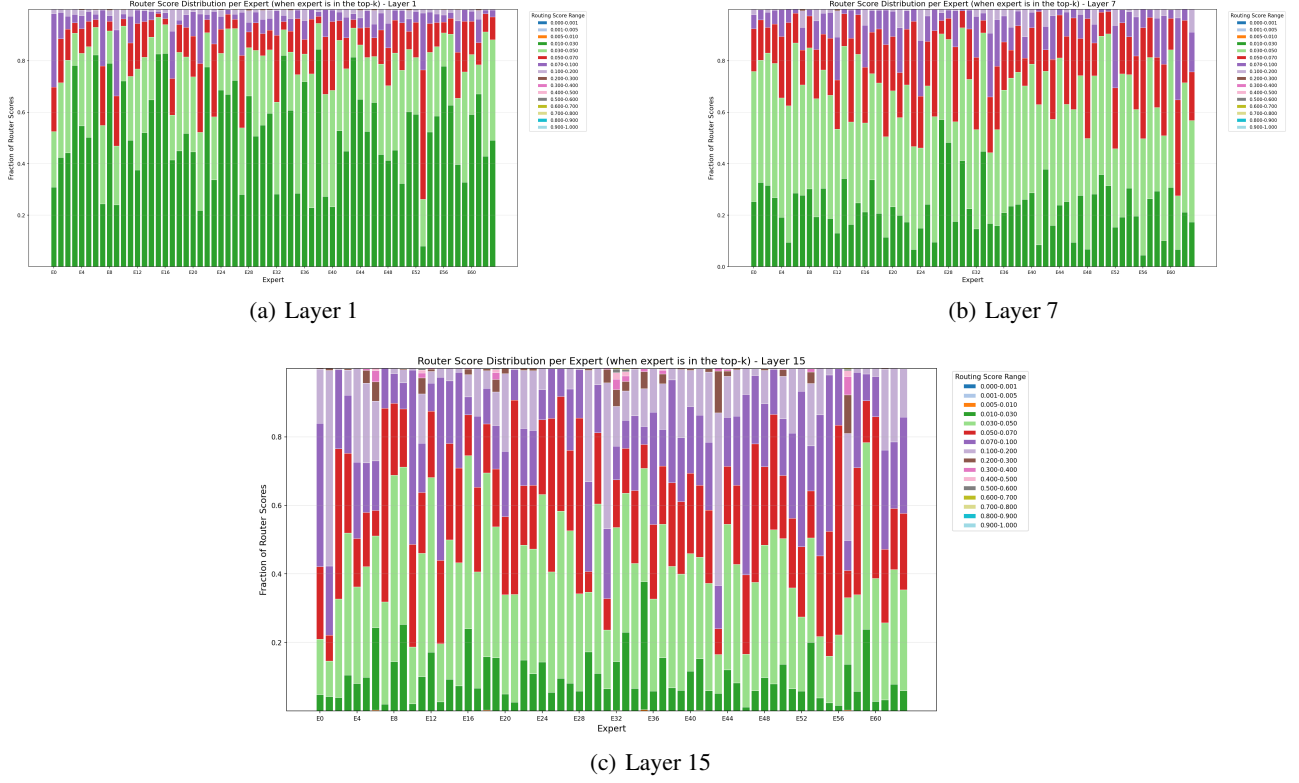


Figure 33. Per-expert router distributions for OLMoE on MMLU at selected layers.

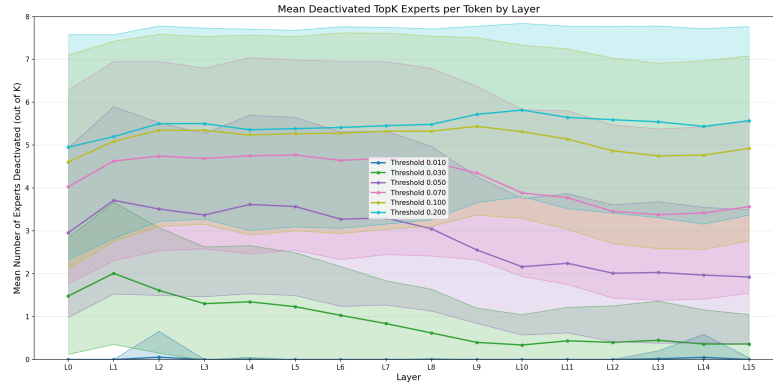
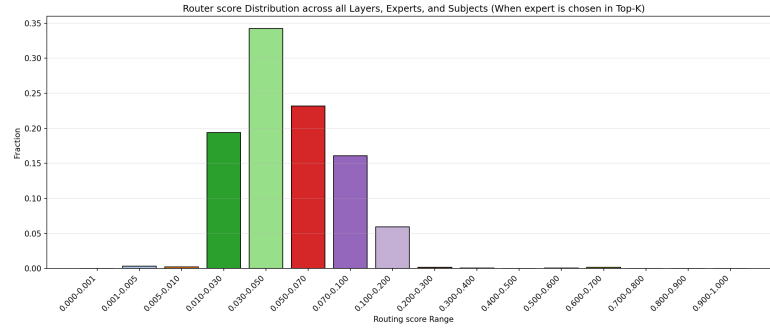
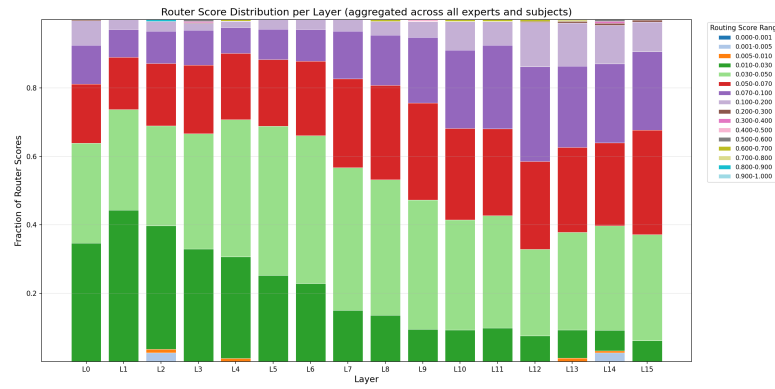


Figure 34. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU.

A.2.2. ALLEN AI ARC



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 35. Router distributions for OLMoE on AllenAI ARC.

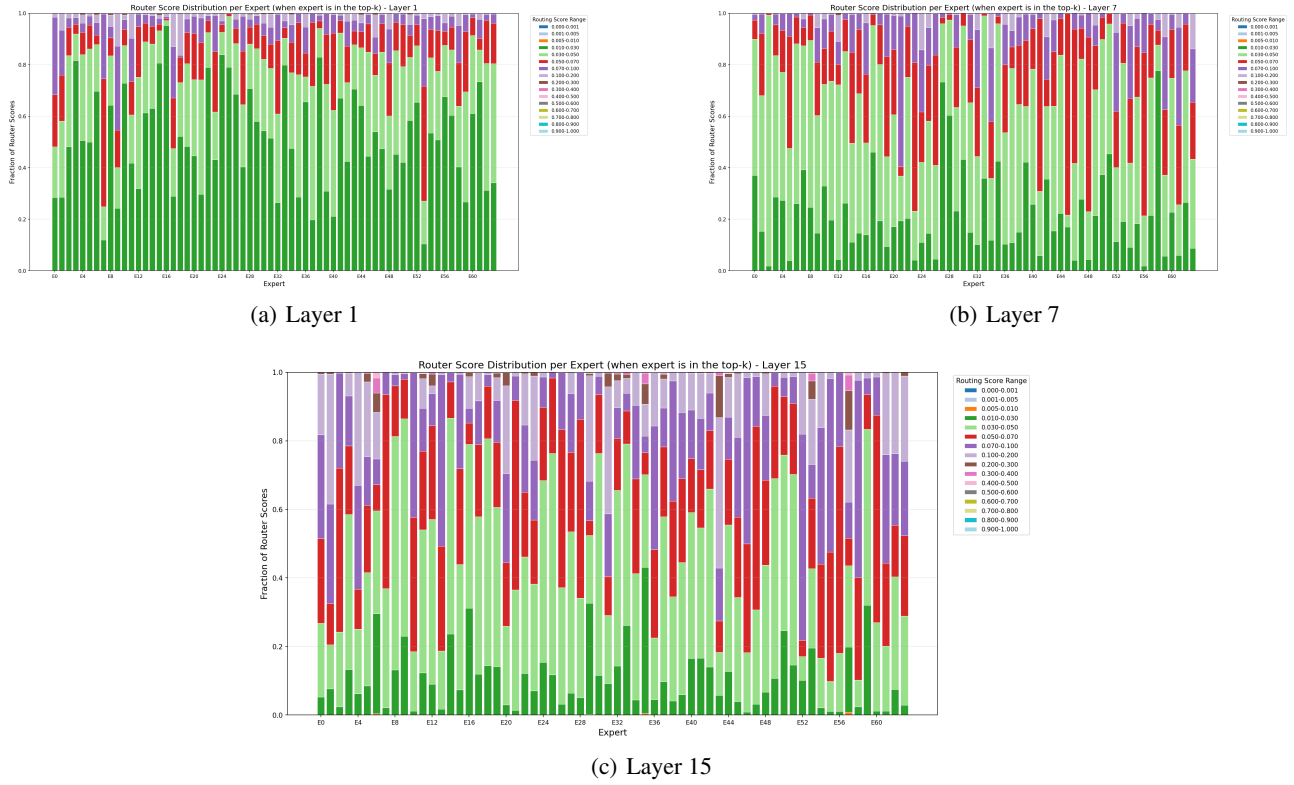


Figure 36. Per-expert router distributions for OLMoE on AllenAI ARC at selected layers.

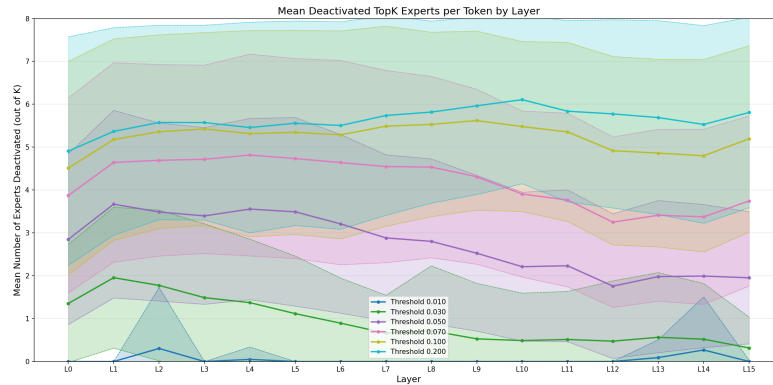
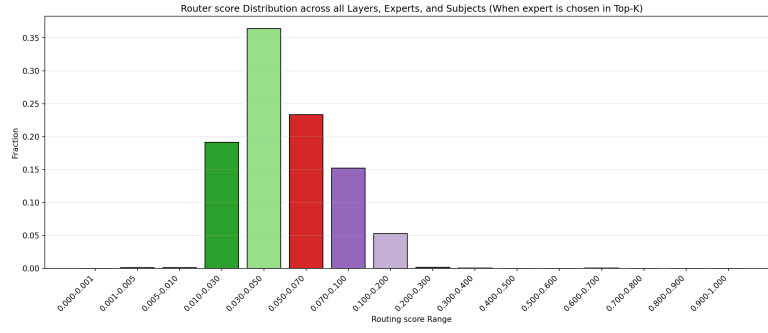
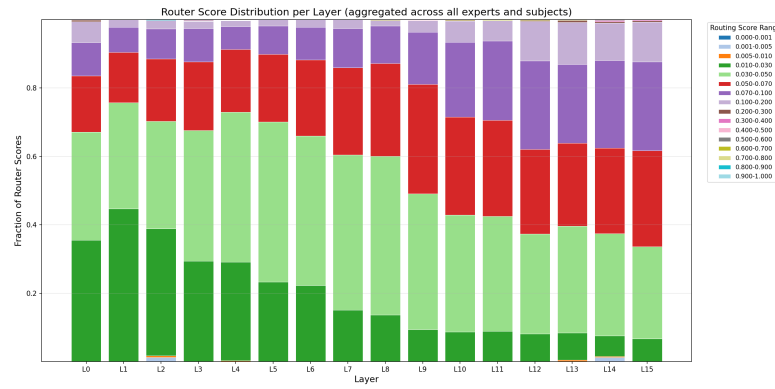


Figure 37. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for AllenAI ARC.

A.2.3. HELLASWAG



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 38. Router distributions for OLMoE on Hellaswag.

MoE Router distribution exploration

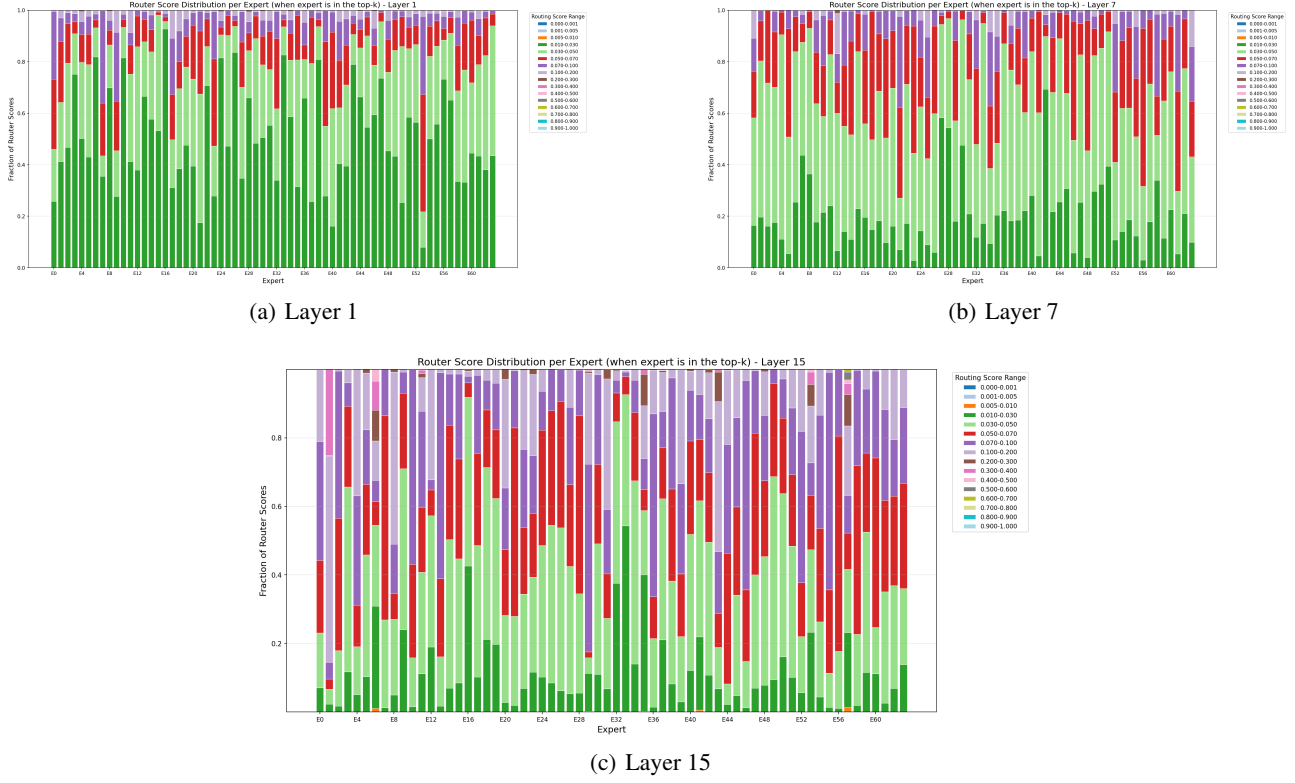


Figure 39. Per-expert router distributions for OLMoE on Hellaswag at selected layers.

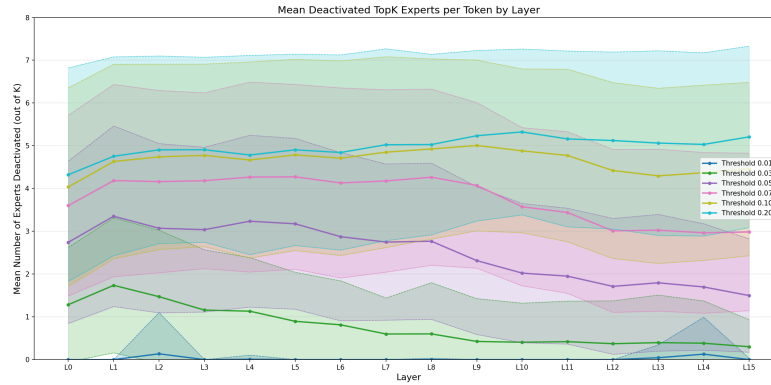
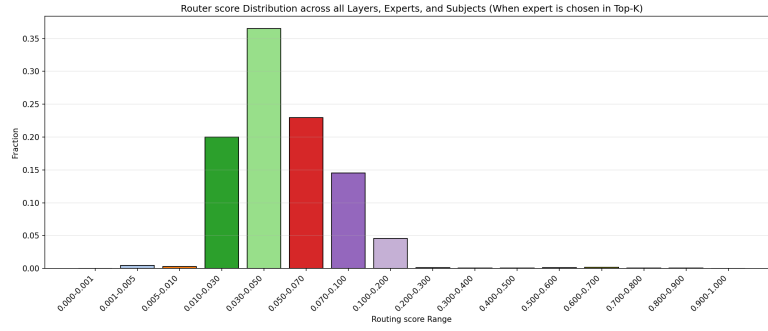
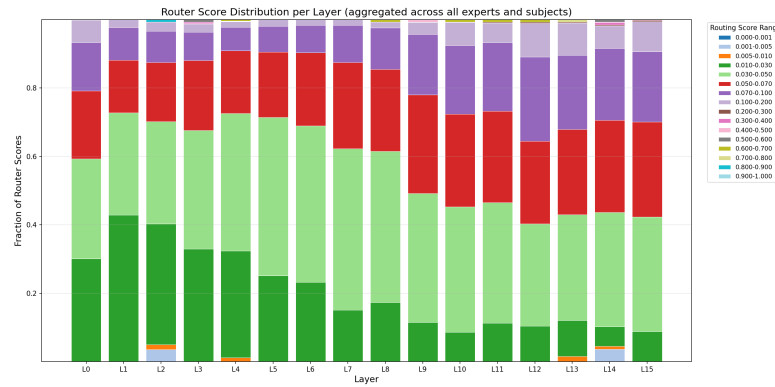


Figure 40. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Hellaswag.

A.2.4. WINOGRANDE



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 41. Router distributions for OLMoE on Winogrande.

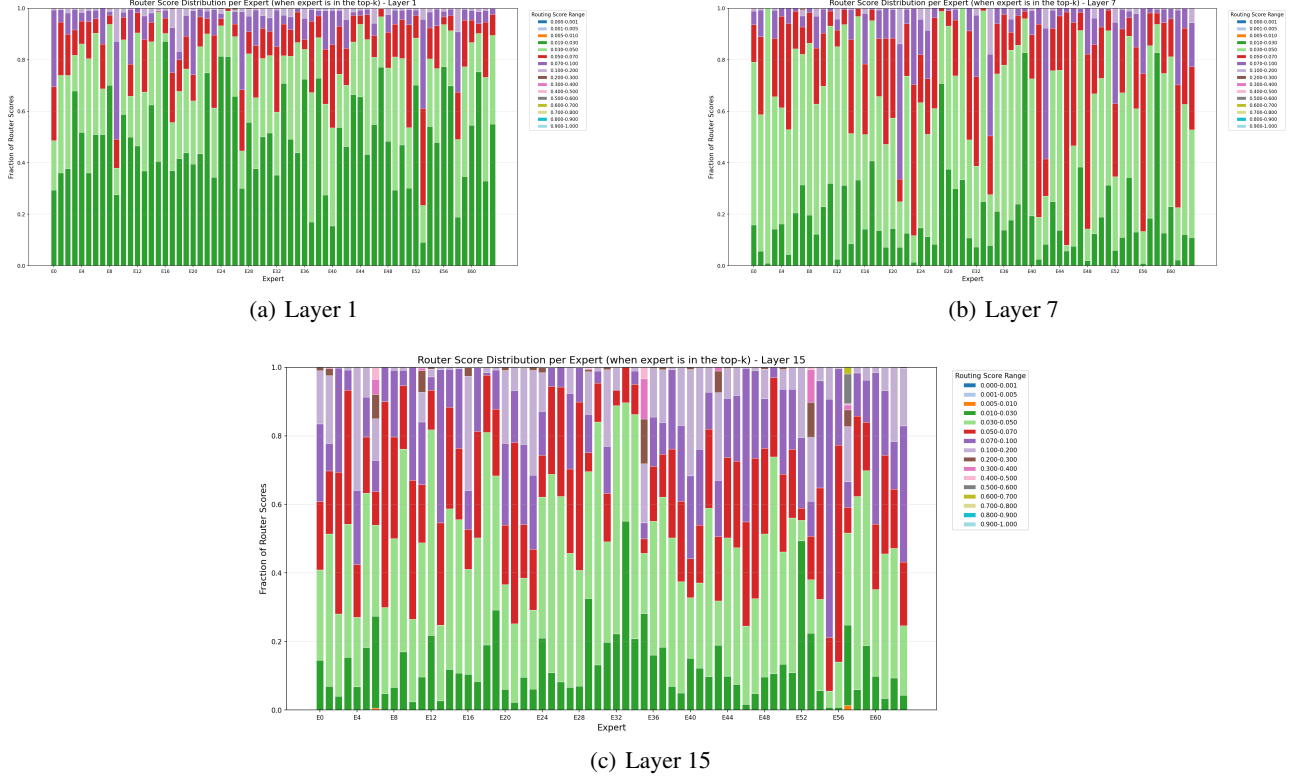


Figure 42. Per-expert router distributions for OLMoE on Winogrande at selected layers.

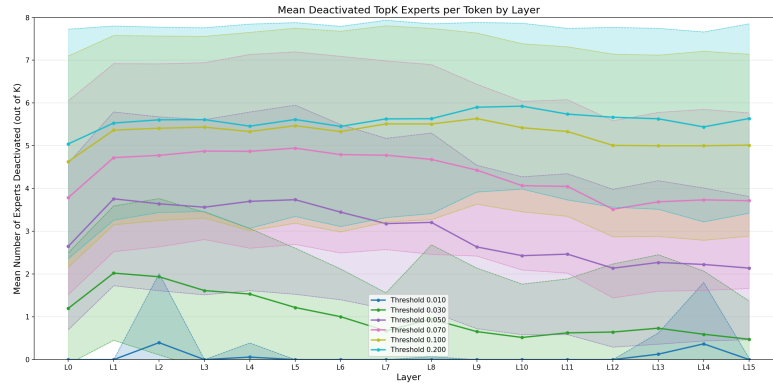
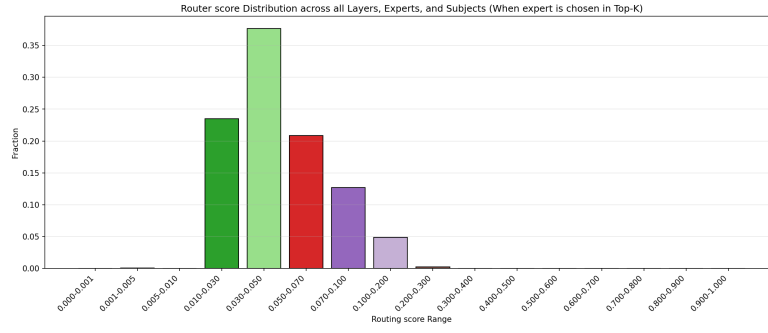
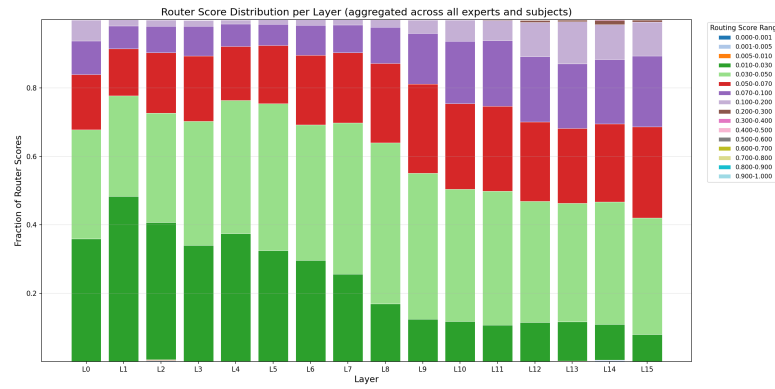


Figure 43. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Winogrande.

A.2.5. MMLU PRO



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 44. Router distributions for OLMoE on MMLU Pro.

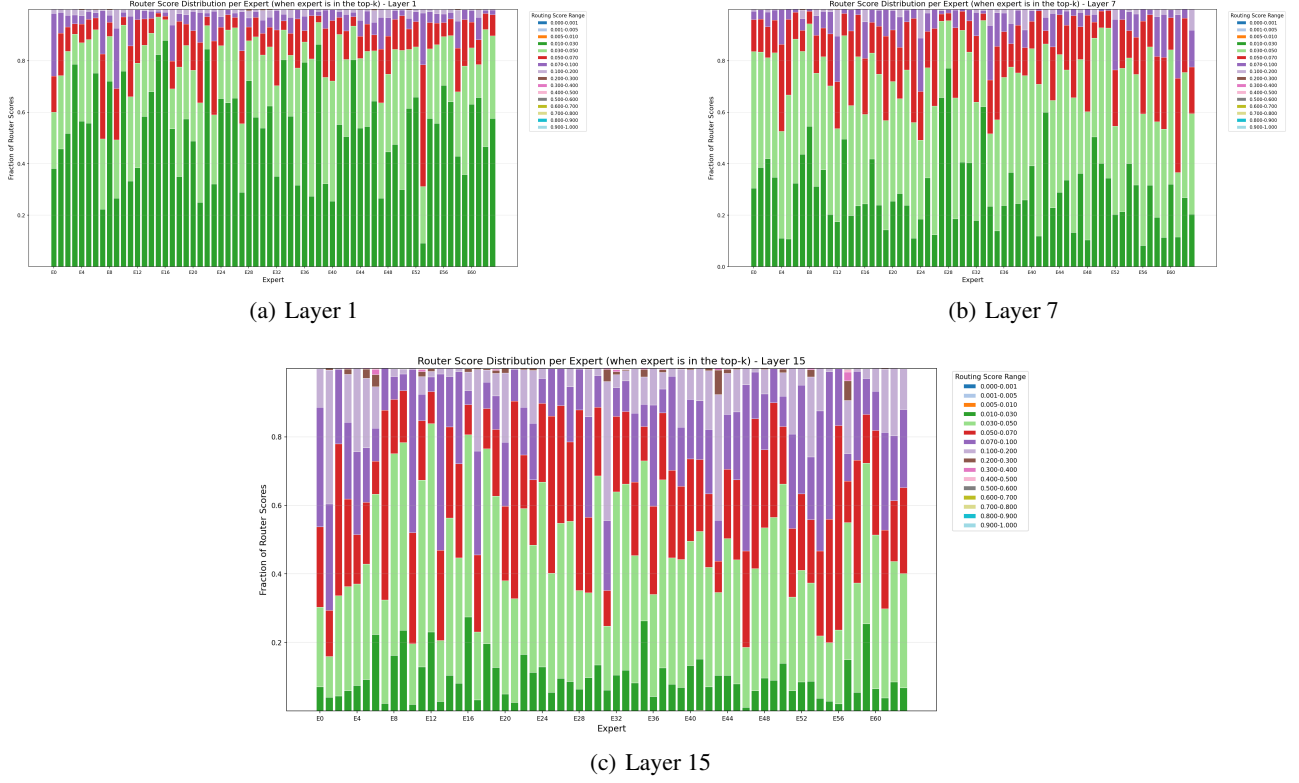


Figure 45. Per-expert router distributions for OLMoE on MMLU Pro at selected layers.

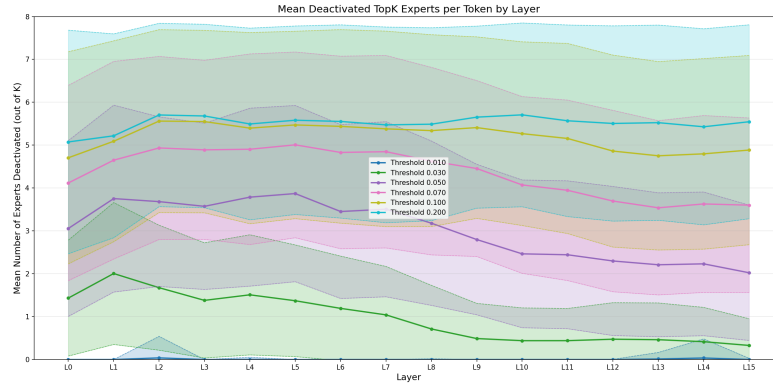
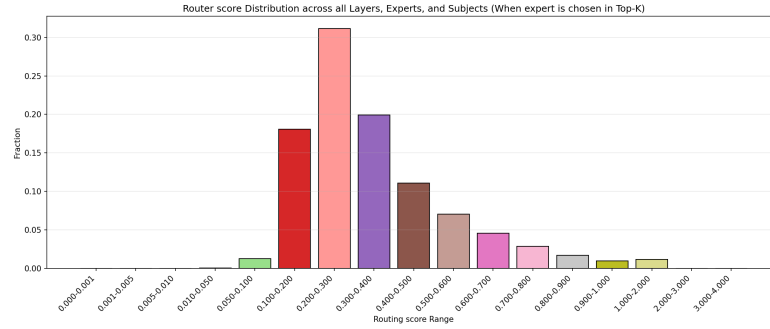


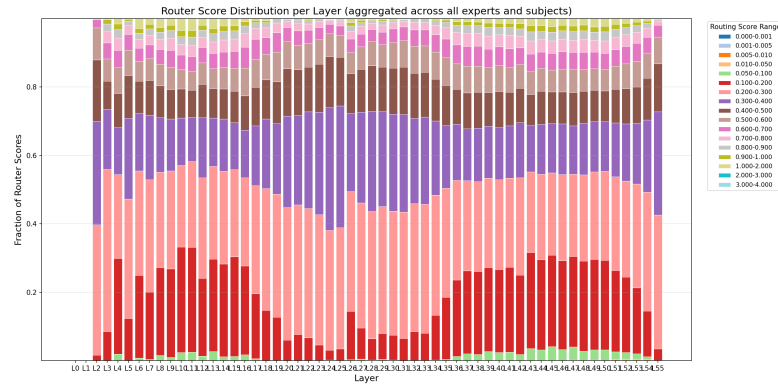
Figure 46. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU Pro.

A.3. Trinity

A.3.1. MMLU



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 47. Router distributions for Trinity on MMLU.

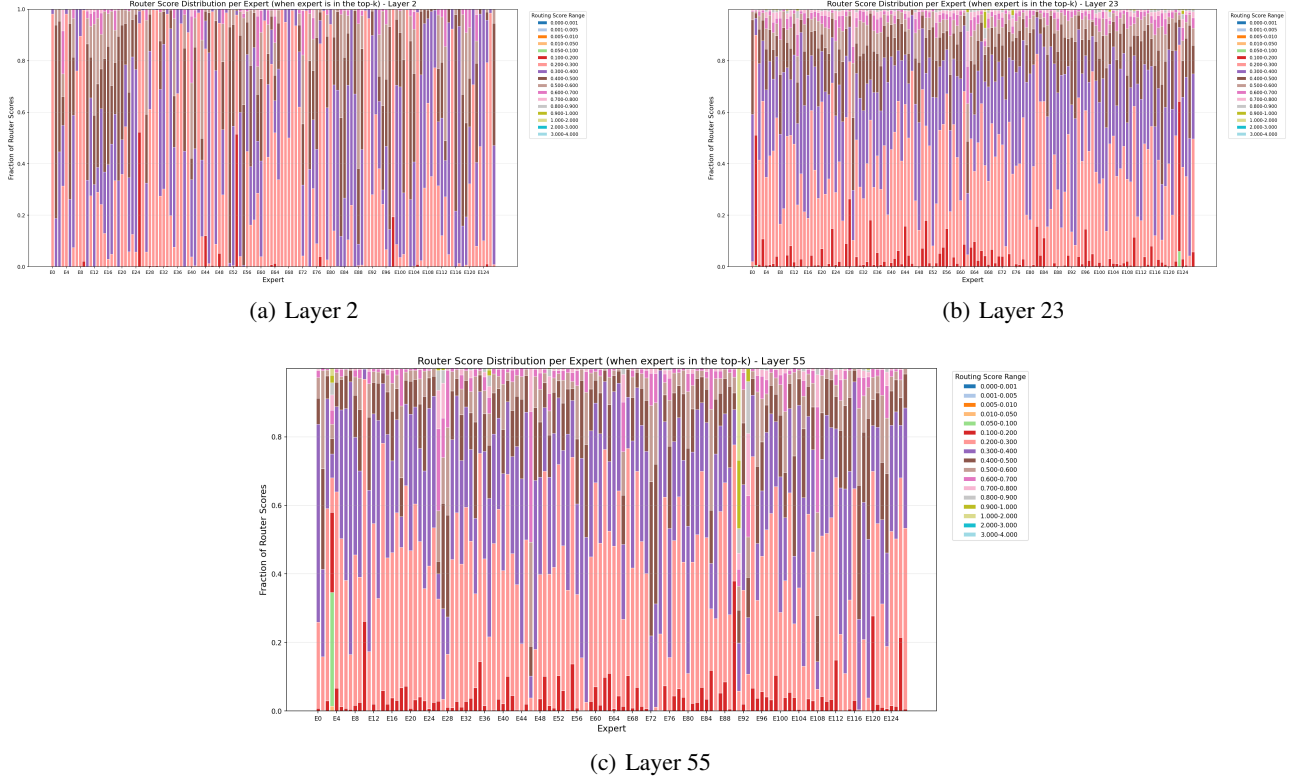


Figure 48. Per-expert router distributions for Trinity on MMLU at selected layers.

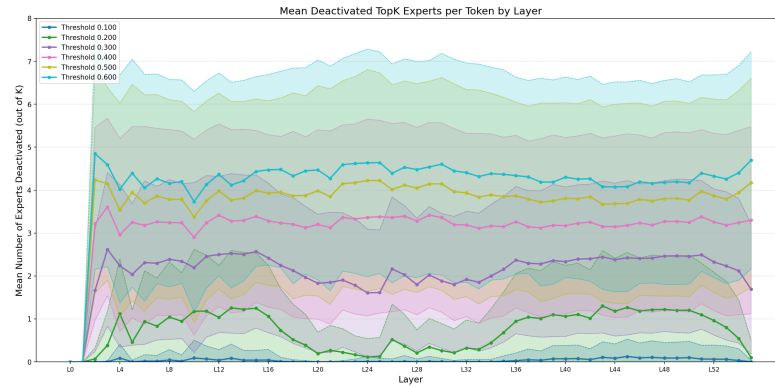
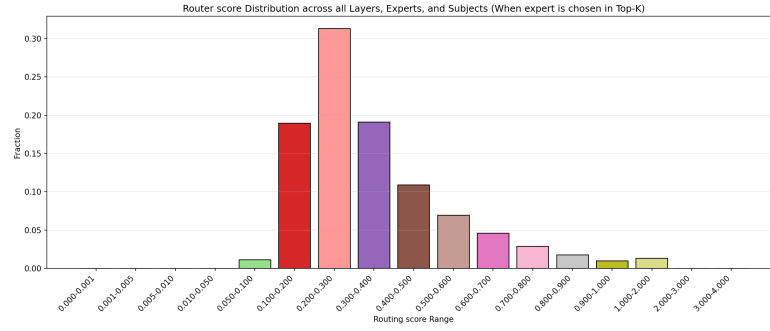
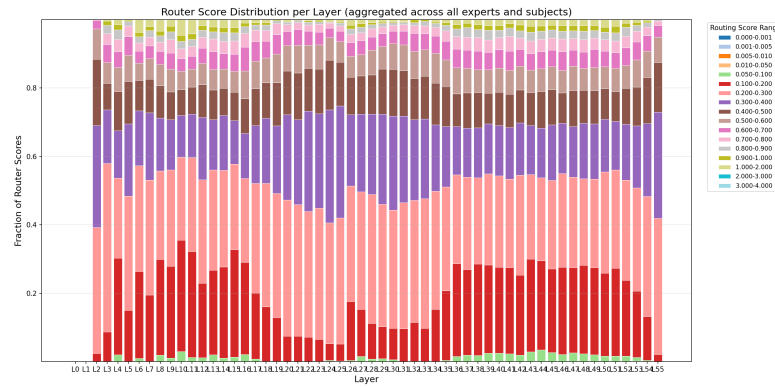


Figure 49. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU.

A.3.2. ALLENAI ARC



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 50. Router distributions for Trinity on AllenAI ARC.

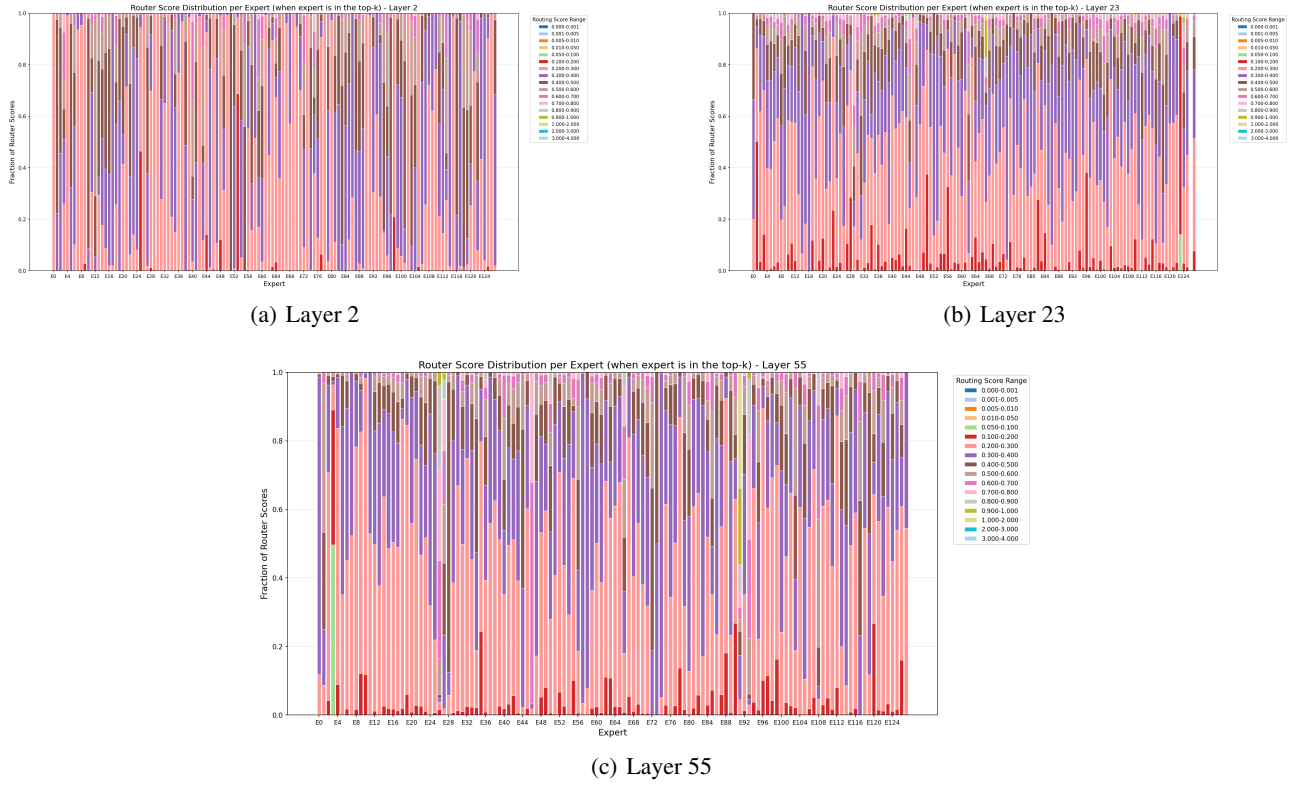


Figure 51. Per-expert router distributions for Trinity on AllenAI ARC at selected layers.

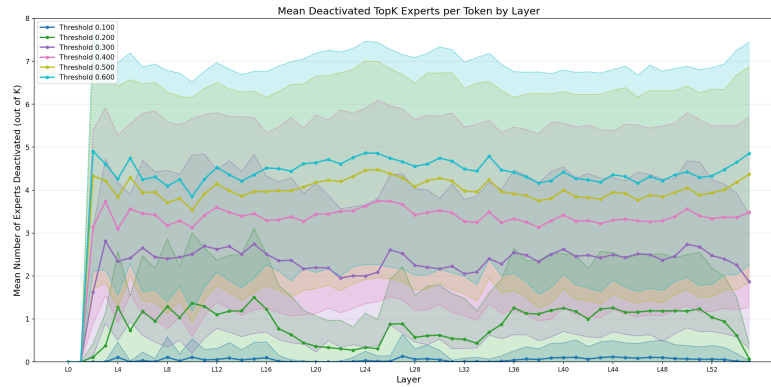
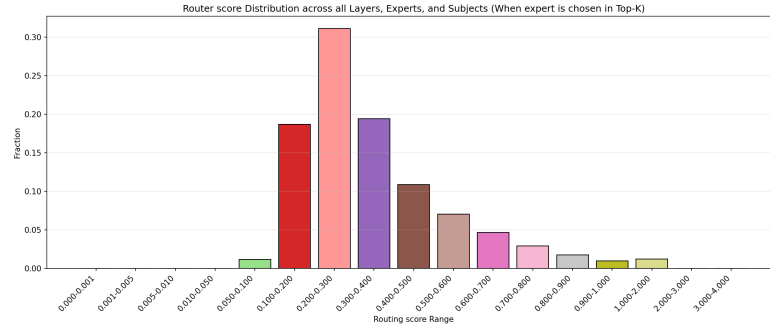
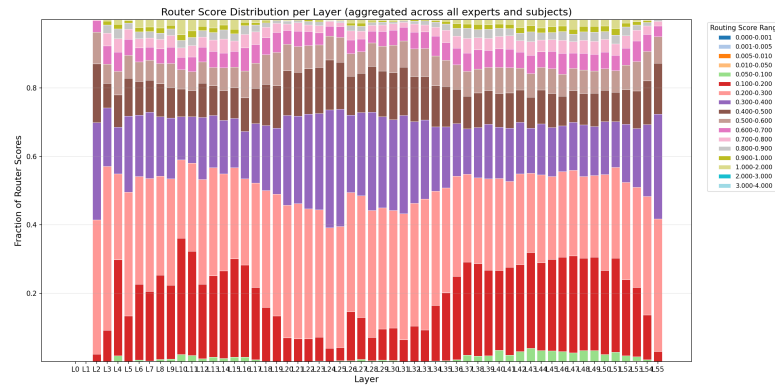


Figure 52. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for AllenAI ARC.

A.3.3. HELLASWAG



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 53. Router distributions for Trinity on Hellaswag.

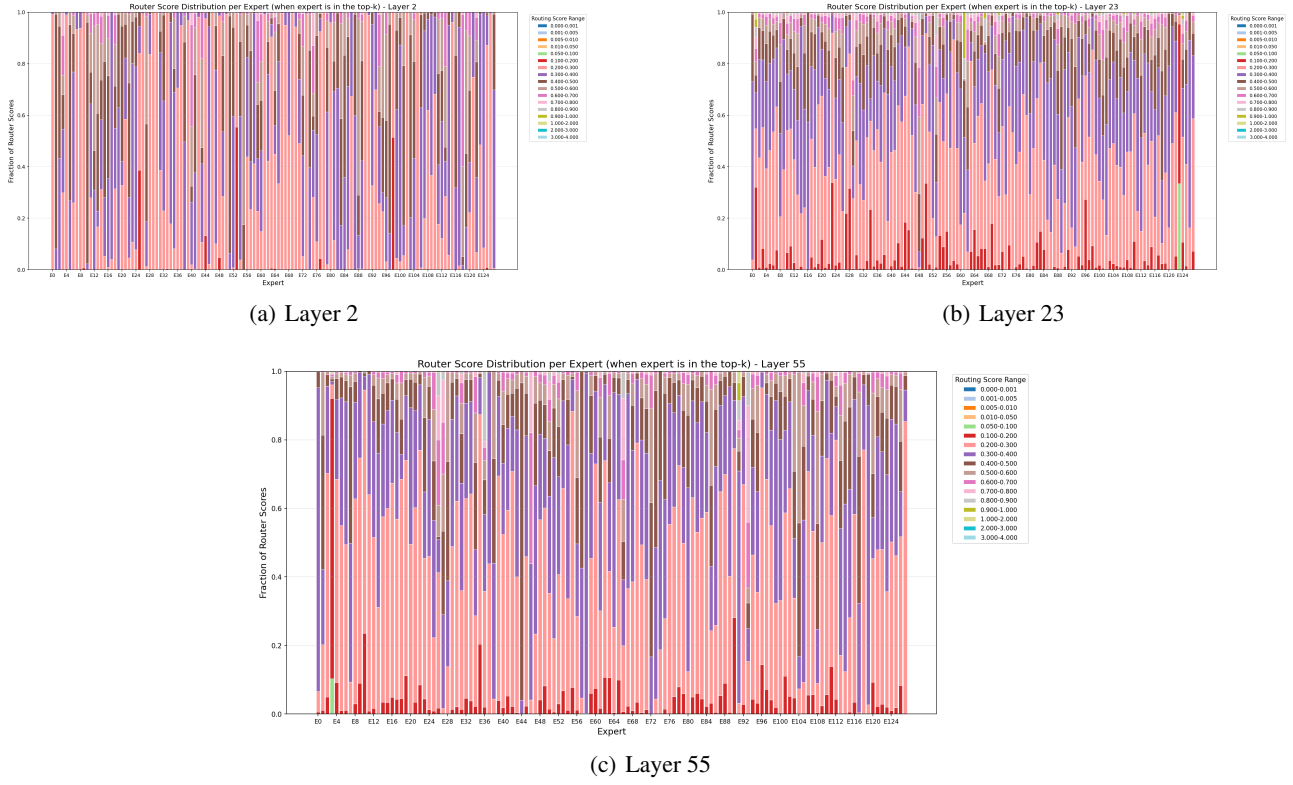


Figure 54. Per-expert router distributions for Trinity on Hellaswag at selected layers.

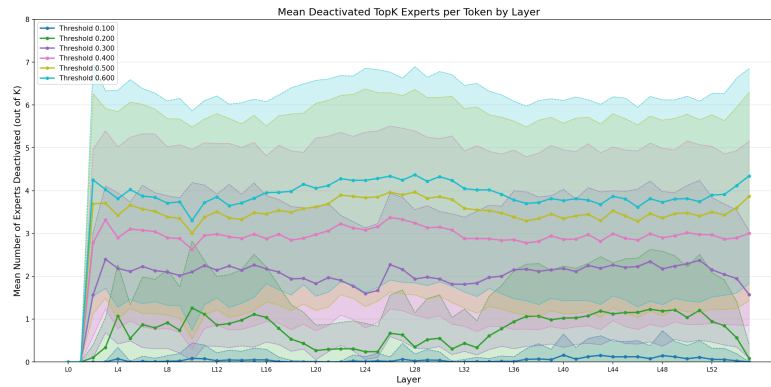
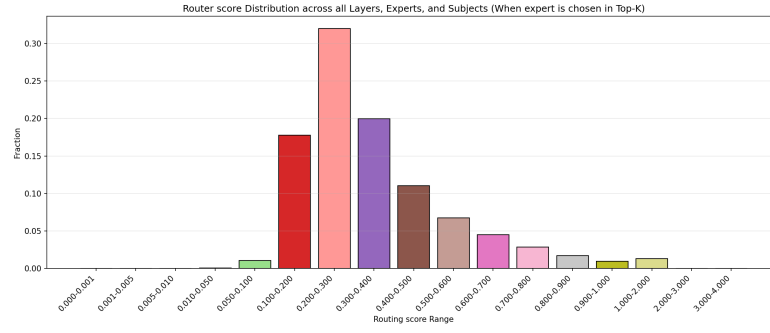
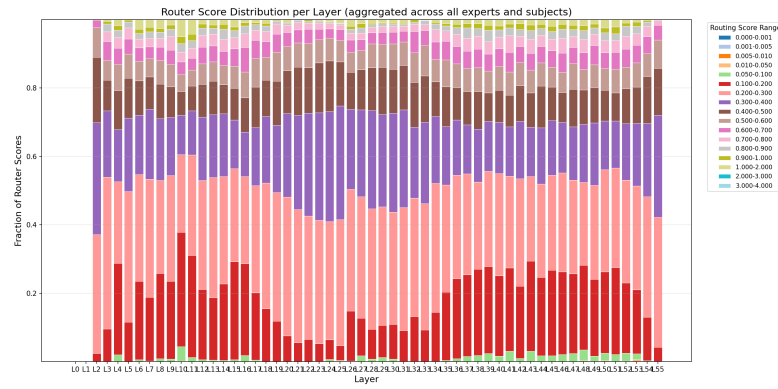


Figure 55. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Hellaswag.

A.3.4. WINOGRANDE



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 56. Router distributions for Trinity on Winogrande.

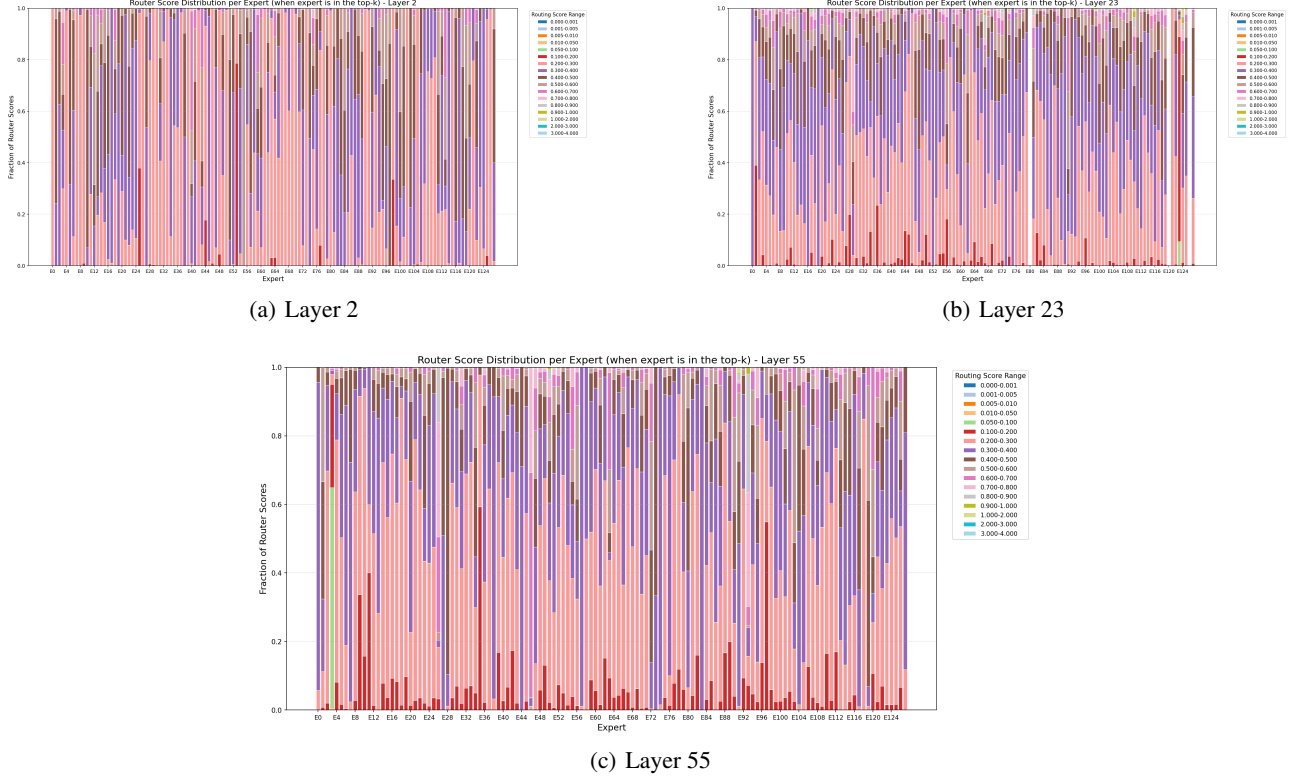


Figure 57. Per-expert router distributions for Trinity on Winogrande at selected layers.

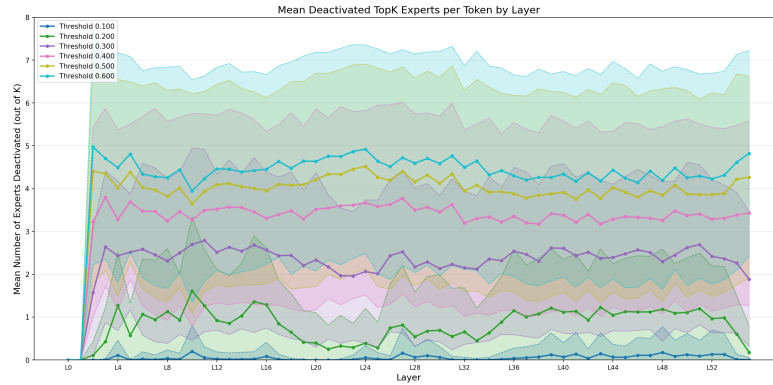
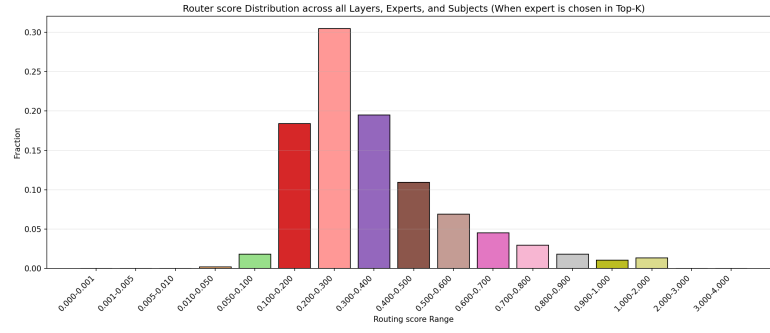
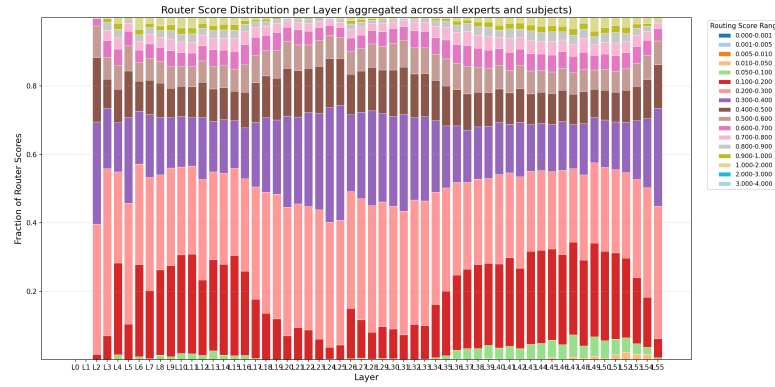


Figure 58. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Winogrande.

A.3.5. MMLU PRO



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 59. Router distributions for Trinity on MMLU Pro.

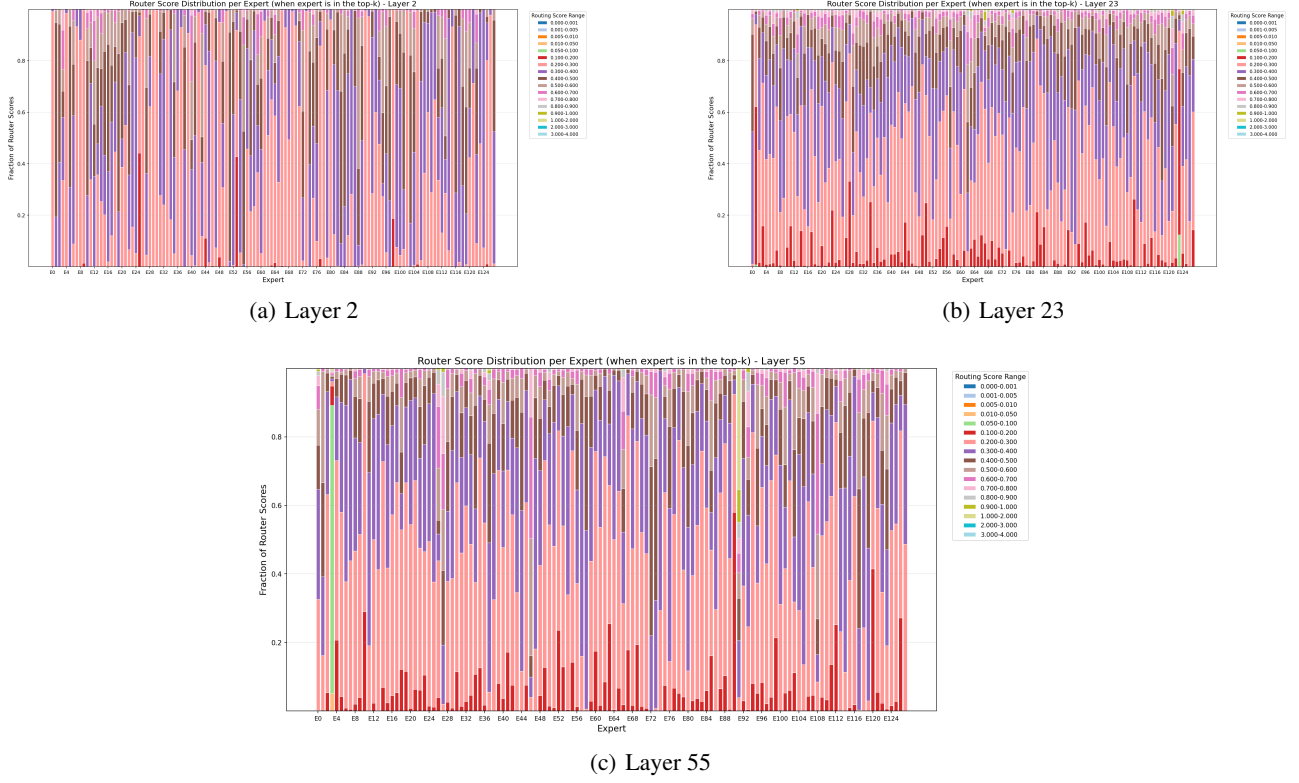


Figure 60. Per-expert router distributions for Trinity on MMLU Pro at selected layers.

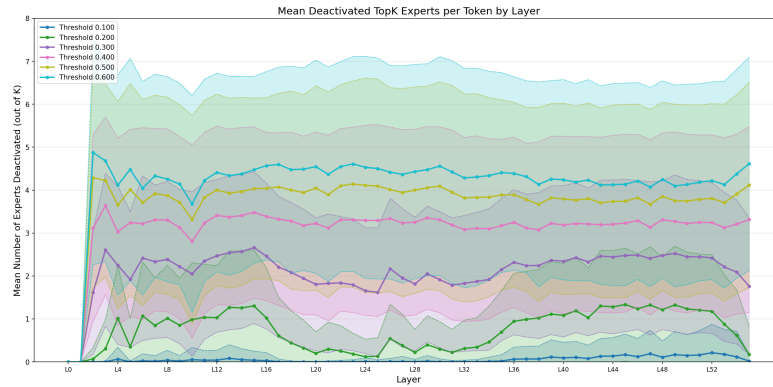
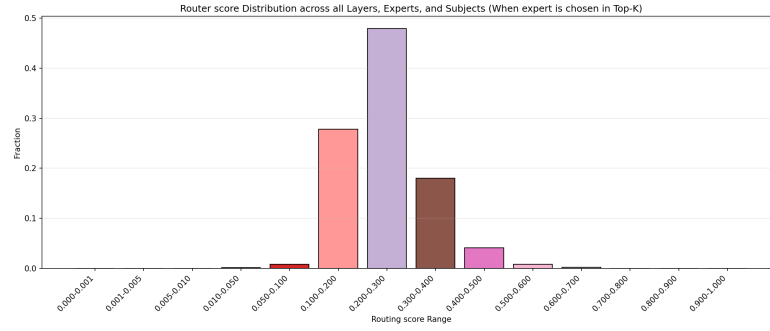


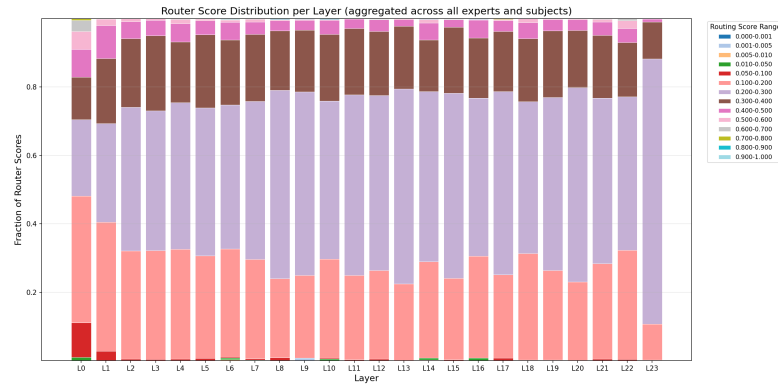
Figure 61. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU Pro.

A.4. GPT-OSS

A.4.1. MMLU



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 62. Router distributions for GPT-OSS on MMLU.

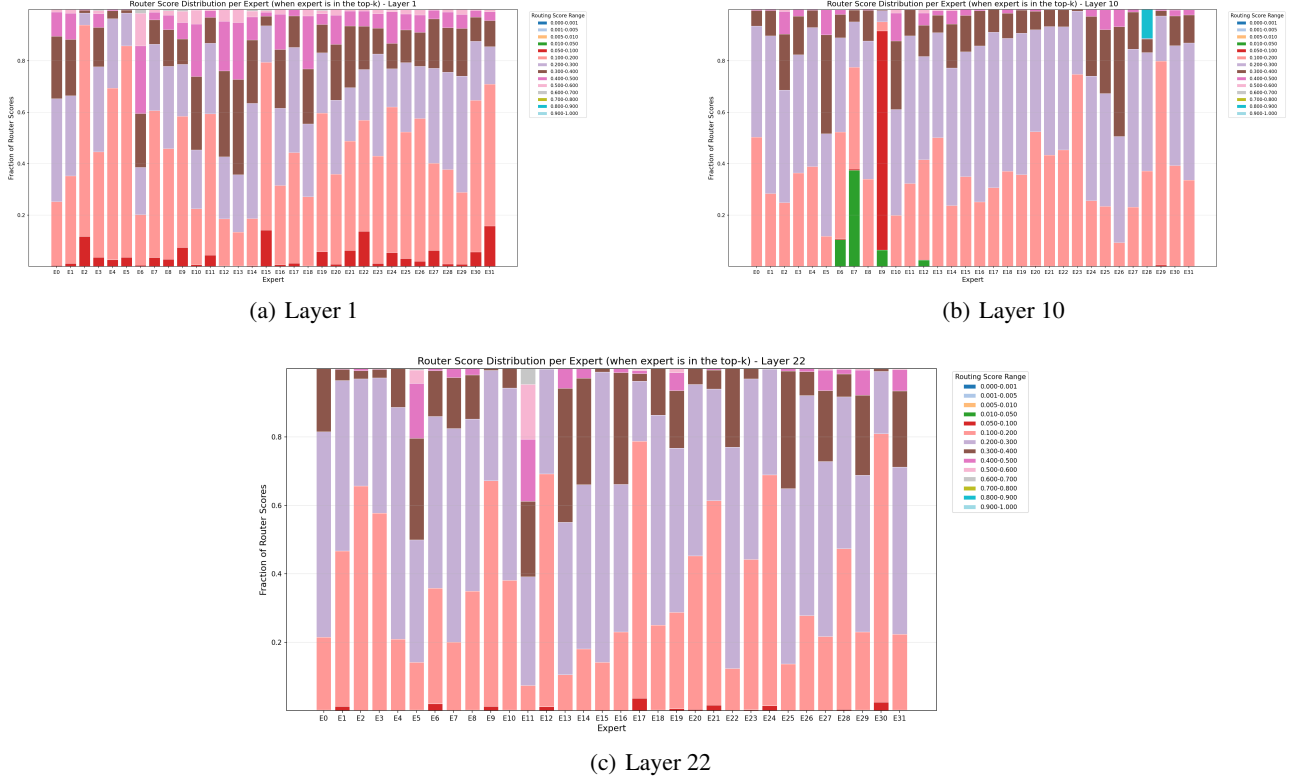


Figure 63. Per-expert router distributions for GPT-OSS on MMLU at selected layers.

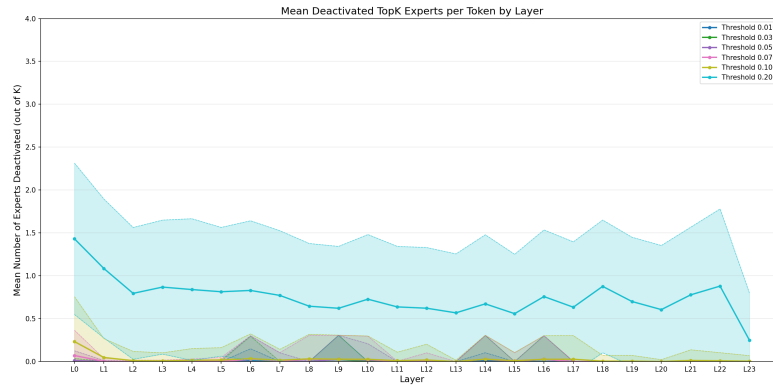
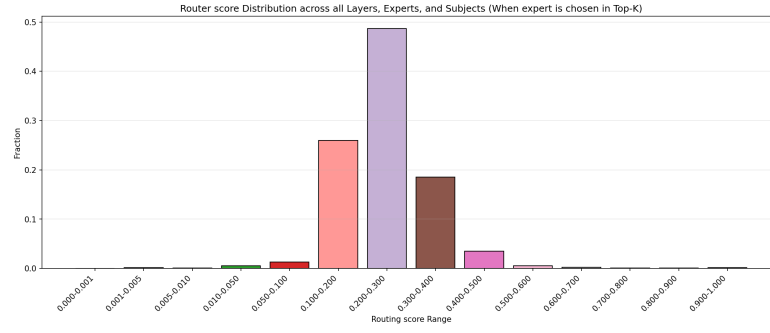
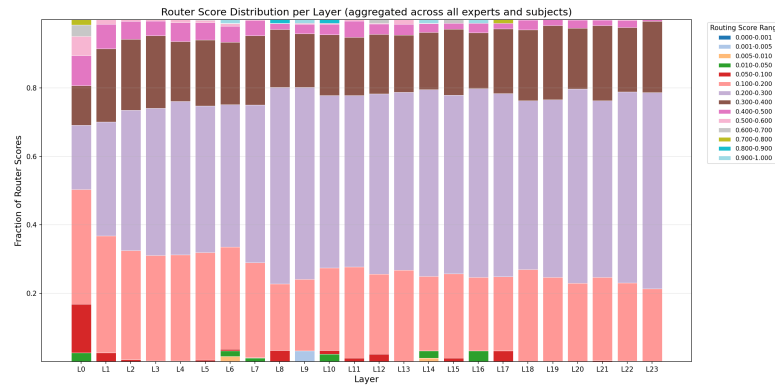


Figure 64. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU.

A.4.2. ALLENAI ARC



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 65. Router distributions for GPT-OSS on AllenAI ARC.

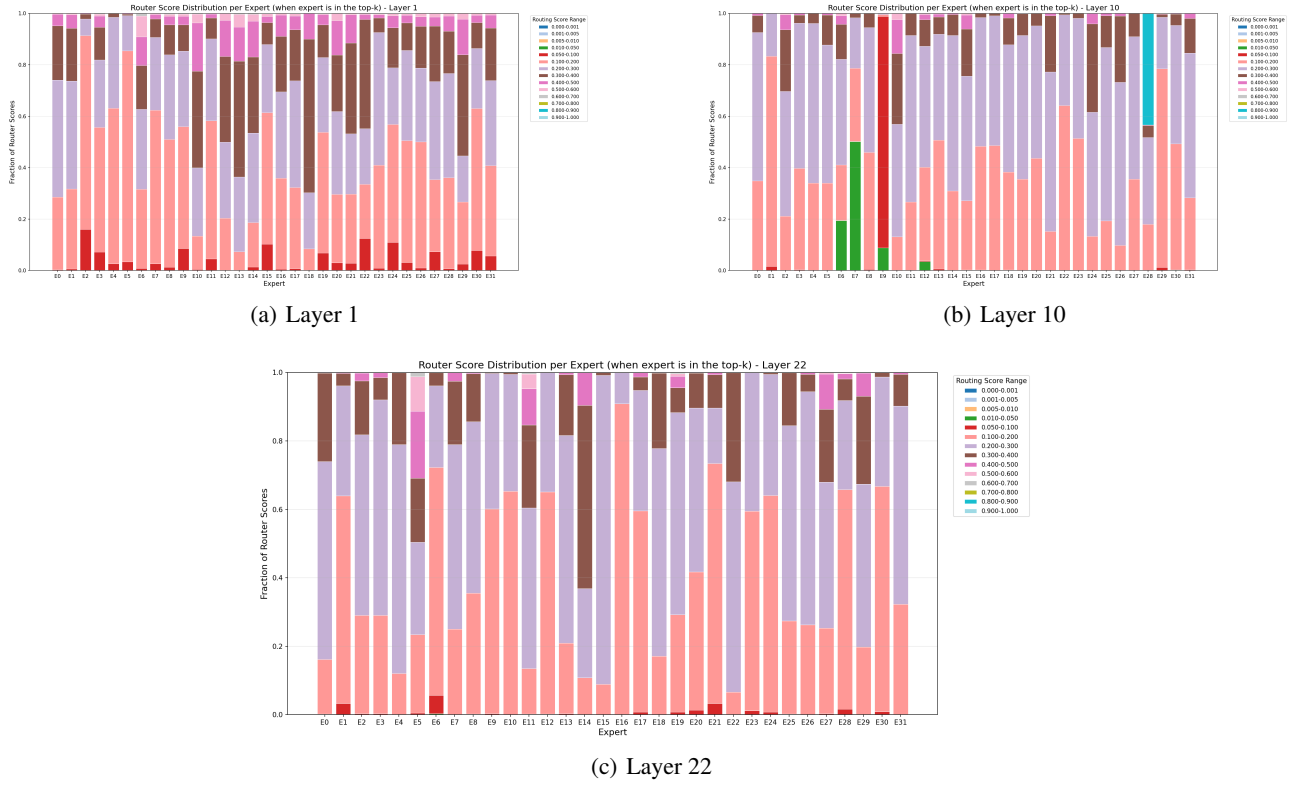


Figure 66. Per-expert router distributions for GPT-OSS on AllenAI ARC at selected layers.

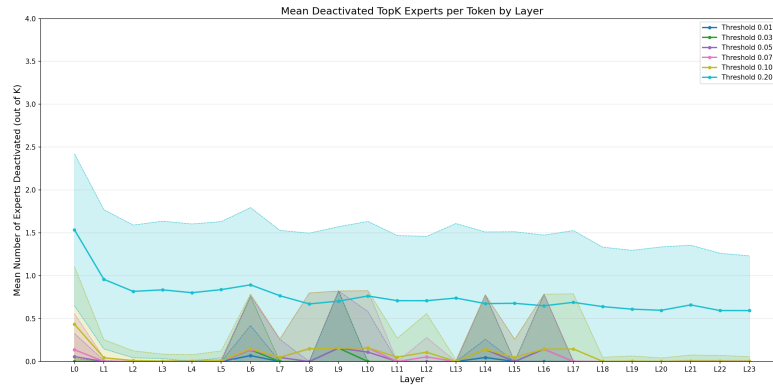
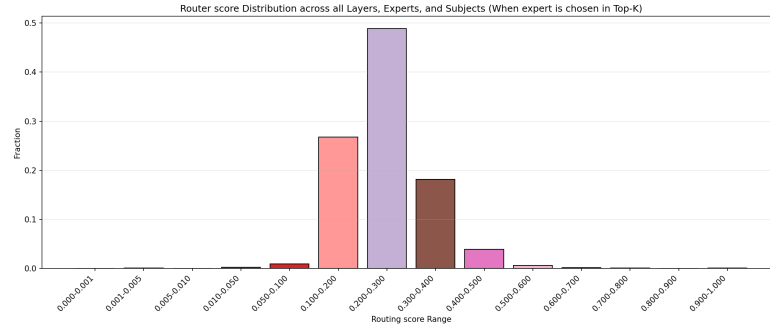
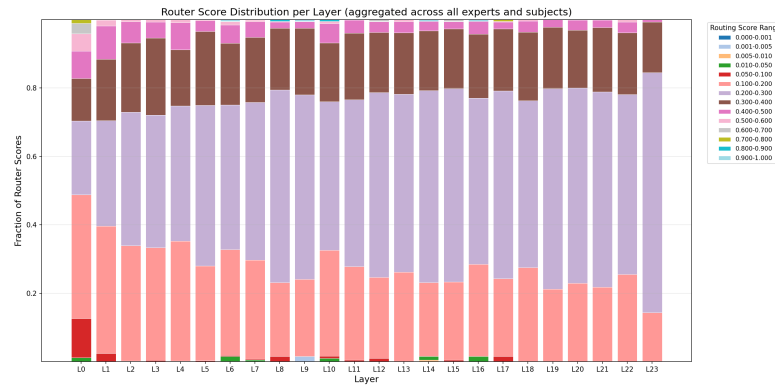


Figure 67. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for AllenAI ARC.

A.4.3. HELLASWAG



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 68. Router distributions for GPT-OSS on Hellaswag.

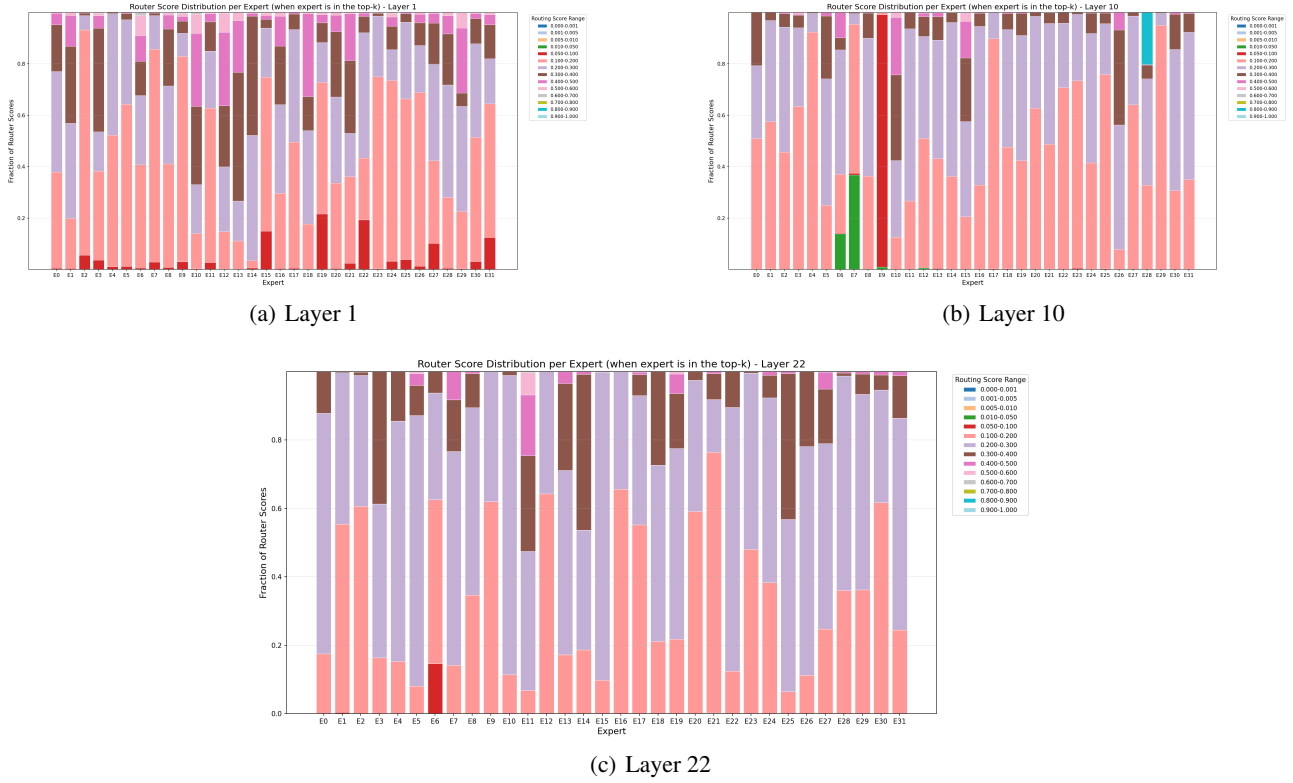


Figure 69. Per-expert router distributions for GPT-OSS on Hellaswag at selected layers.

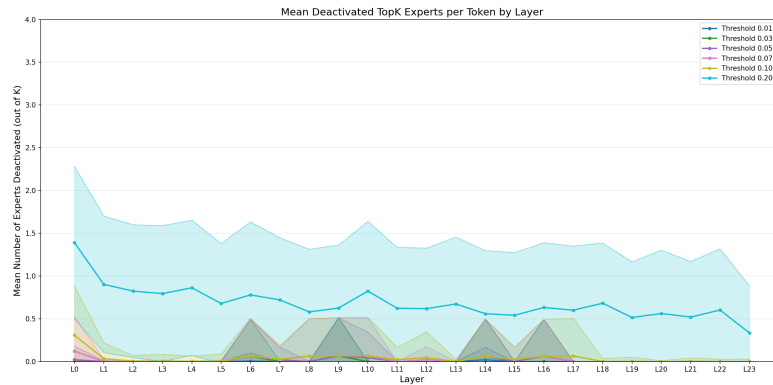
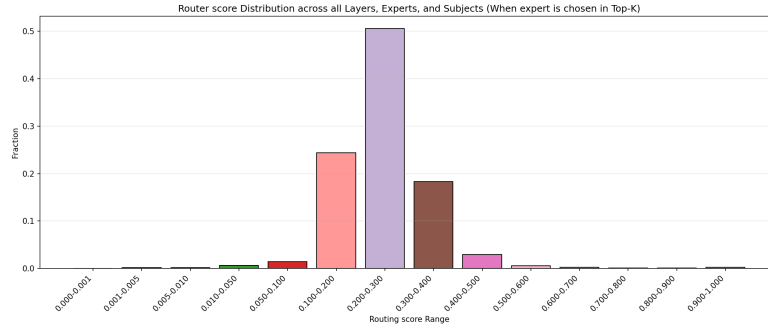
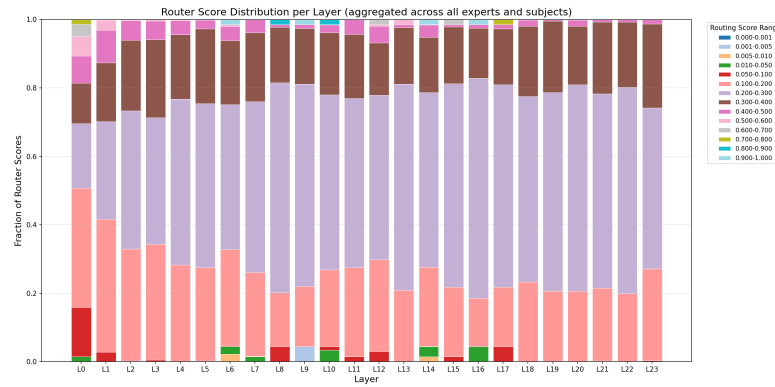


Figure 70. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Hellaswag.

A.4.4. WINOGRANDE



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 71. Router distributions for GPT-OSS on Winogrande.

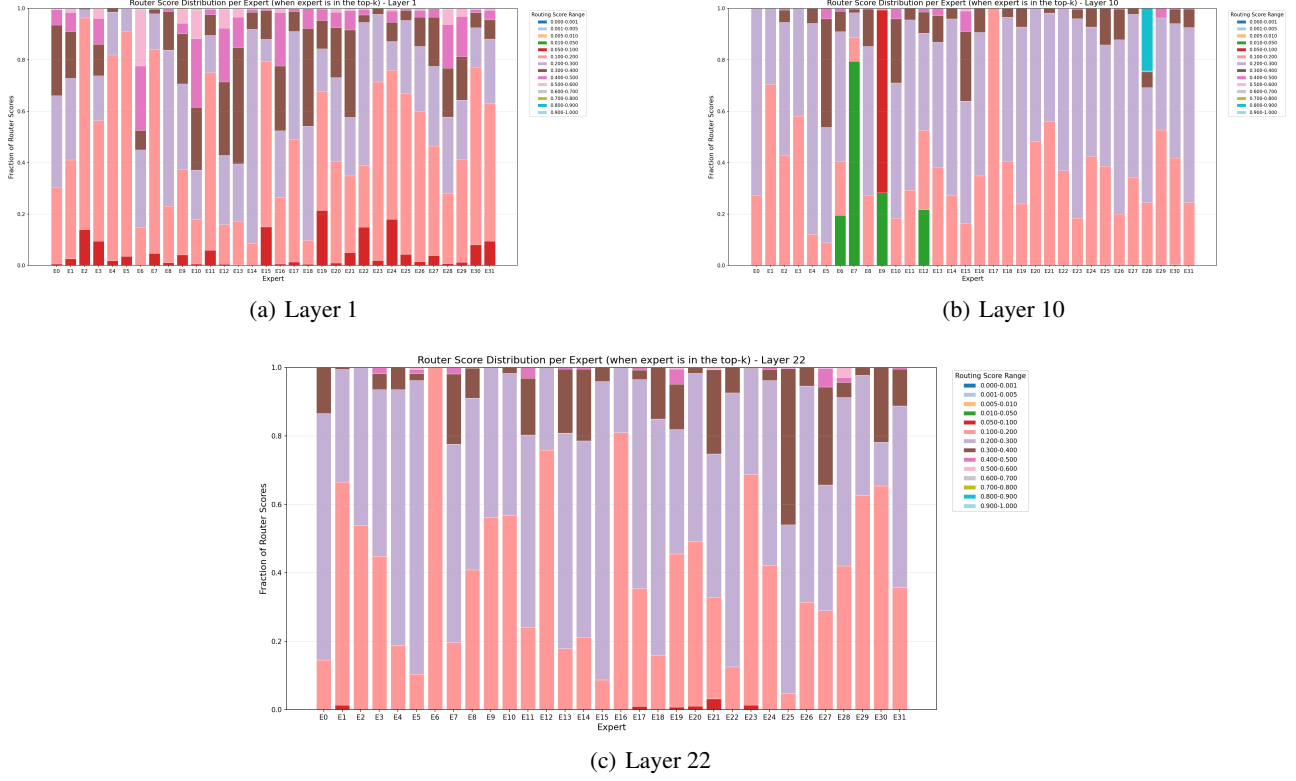


Figure 72. Per-expert router distributions for GPT-OSS on Winogrande at selected layers.

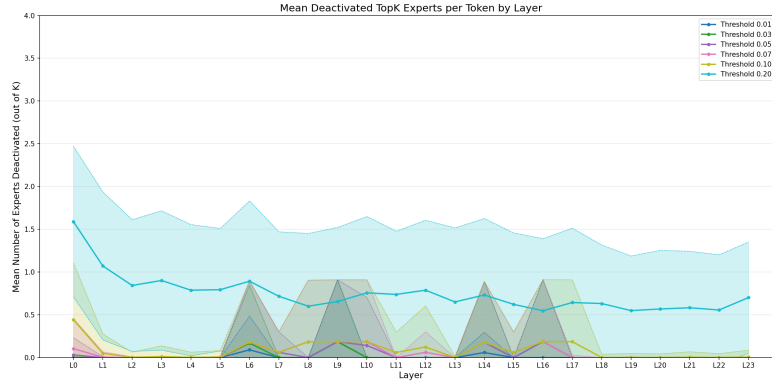
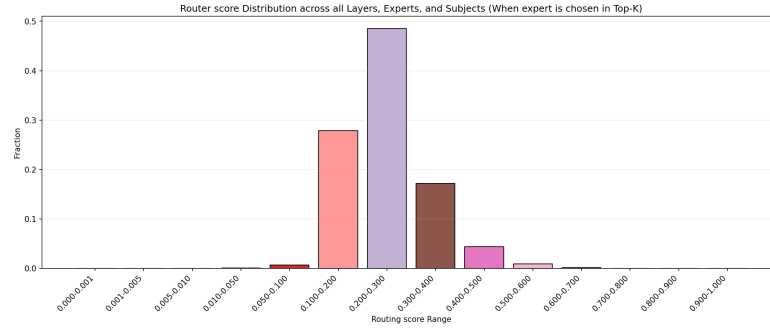
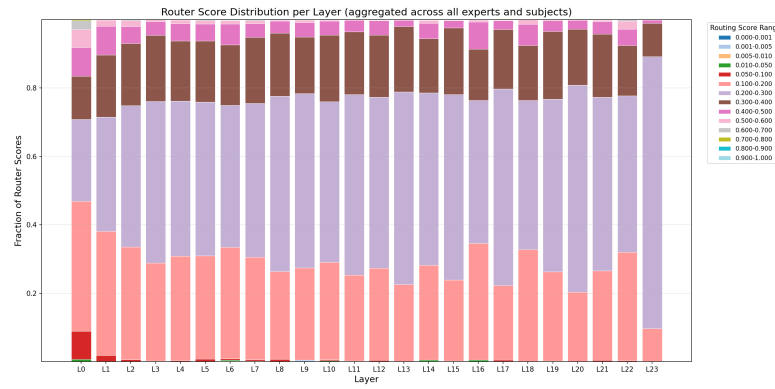


Figure 73. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for Winogrande.

A.4.5. MMLU PRO



(a) Global router distribution. This shows all the Tokens · Top-k that were created by the MoE from all the layers for all tokens in its chosen K experts.



(b) Per-layer router distribution

Figure 74. Router distributions for GPT-OSS on MMLU Pro.

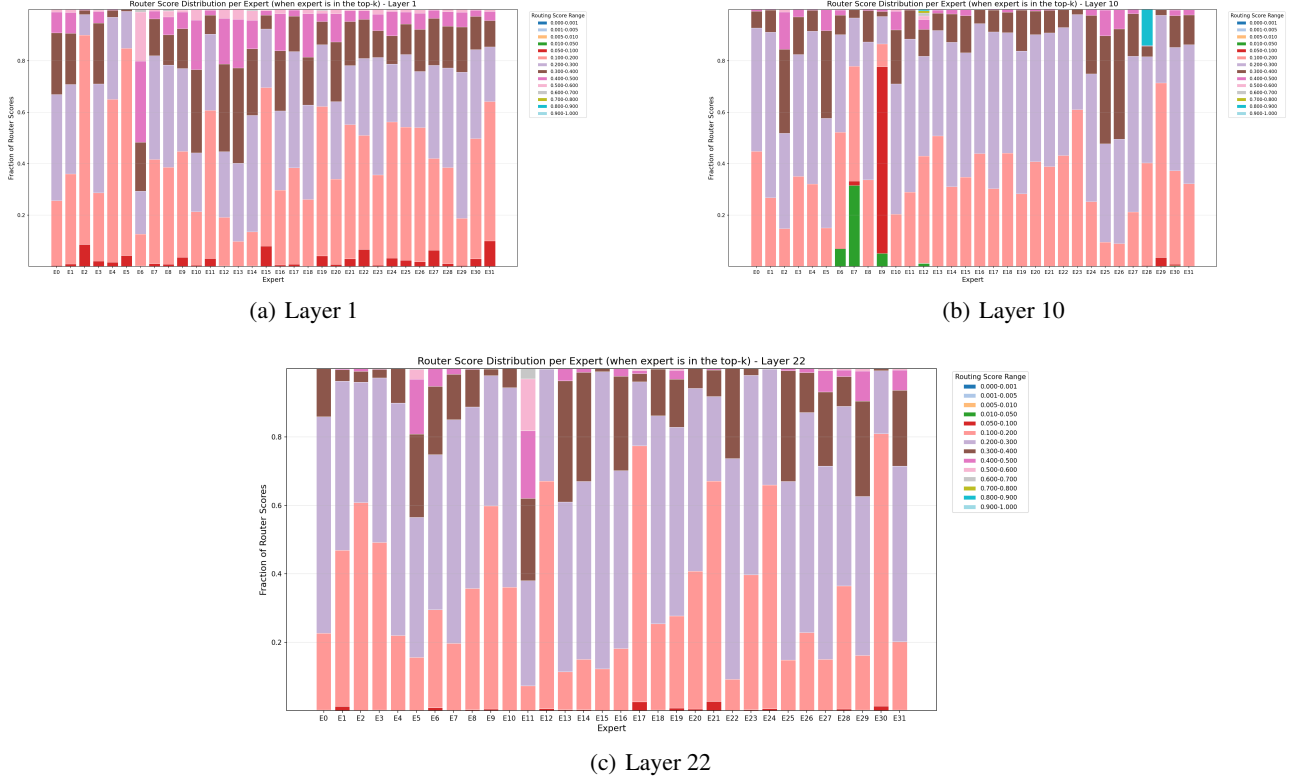


Figure 75. Per-expert router distributions for GPT-OSS on MMLU Pro at selected layers.

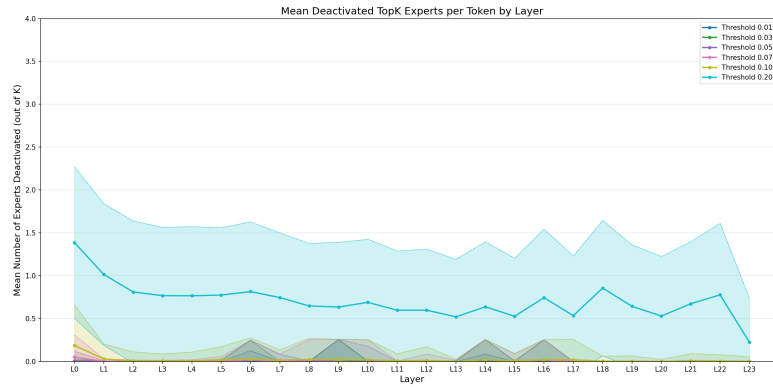


Figure 76. Here we show how many experts on average (and its standard deviation) would be deactivated if we chose a threshold p for MMLU Pro.